UNIVERSITI TUNKU ABDUL RAHMAN

MAY 2024 TRIMESTER

ASSIGNMENT

REPORT

Candidate is required to fill in ALL the information below:

Group Leader: (*Name, ID, Programme*)	**Kyle Yamato Christian, (2302452), SE**		
Group member 1: (*Name, ID, Programme*)	**Tan Jun Yin, (2302533), SE**		
Group member 2: (*Name, ID, Programme*)	**Ang Yong Xen, (2301521), SE**		
Group member 3: (*Name, ID, Programme*)	**Lim Jun Sheng, (2302532), SE**		
Faculty /Institute/ Centre:	**LKC FES**	Group number:	**G5**
Course Code:	**UECS1004/UECS1104**	Course Description:	**Programming and Problem Solving**

Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

DECLARATION STATEMENT

We hereby solemnly and sincerely declare and confirm that we have read, understood and shall abide and comply with all laws, rules, regulations, guidelines and lawful instruction of the University and its staff in relation to the commencement of any assessment / examination during my programme of study in Universiti Tunku Abdul Rahman.

We hereby declare that our submission for all assessment / examination during our programme of study in the University shall be based on our original work, not plagiarised from any source(s) except for citations and quotations which have been duly acknowledged. We are fully aware that students who are suspected of violating this pledge are liable to be referred to the Examination Disciplinary Committee of the University.

We further certify that we have read and understand the above guidelines and agree to abide by the above declarations. The group leader signs this declaration statement on behalf of any and all group members.

_____

Group Leader and Member Names and Student I.D.:

1) Kyle Yamato Christian (2302452)
2) Tan Jun Yin (2302533)
3) Ang Yong Xen (2301521)
4) Lim Jun Sheng (2302532)

Date: 31 August 2024

TABLE OF CONTENTS

CHAPTER

1 INTRODUCTION TO SOLUTION DESIGN

1.1 Introduction/Background

As the modern work landscape evolves, remote work has become a vital component of many organizations. Ensuring that employees have safe, ergonomic, and productive home office setups is crucial for their well-being and efficiency. This project addresses the need for a systematic approach to assess and document the safety and suitability of home office environments.

The proposed solution involves a software system designed to help organizations and employees evaluate home workspaces. It includes a detailed checklist covering various aspects such as ergonomics, electrical safety, and environmental factors. The system guides users through a series of questions, documents their responses, and generates comprehensive reports assessing workspace suitability. These reports provide a summary indicating whether the workspace is fully suitable, mostly suitable, or in need of improvements.

Additionally, the system features a productivity tracking component that allows employees to record work activities, assess their productivity, and generate detailed reports. This ensures both the safety of the work environment and the efficiency of work activities are systematically monitored and documented.

1.2 System Roles/Users

The Remote Work Readiness and Productivity Tracking System supports two primary roles: Employees and Managers/Supervisors, each with specific responsibilities and interactions.

Employees:
Employees are the main users and use the system to ensure their home workspace meets safety standards and to track productivity. Their key tasks include:

- **Completing Safety Checklists:** Employees answer a series of 22 safety-related questions to evaluate their home office readiness. Their responses generate safety reports that help identify potential hazards.
- **Logging Work Activities:** Employees record their daily tasks and the time spent on each, allowing the system to monitor and record employee productivity.
- **Generating Reports:** Employees can review reports based on their safety assessments and productivity logs, gaining insights into their work environment and performance.

Managers/Supervisors:
Managers/Supervisors are responsible for overseeing the safety and productivity of their team members. Through these actions, managers ensure that employees' home offices meet safety standards and maintain productivity levels. Their main duties include:

- **Reviewing Safety Reports:** Managers assess the safety checklist reports submitted by employees to identify potential hazards or areas for improvement in home work environments.

- **Monitoring Productivity:** The manager can analyse employees' productivity reports and gauge the employees' efficiency.

1.3 System Modules

*1.3.1 Module 1: **Main Menu***

The **Main Menu Module** is managed by the **mainmenu()** function and acts as the central hub of the system, directing users to the different available features. When an employee launches the program, they are presented with the main menu, which offers three choices: 1) Remote Work Readiness, 2) Productivity Tracking, and 3) Exit. The employee selects an option by entering the corresponding number. If the employee chooses the first option, the system moves to the Remote Work Readiness module by calling the **remoteWorkReadiness()** function, where they can evaluate the suitability of their home workspace. Selecting the second option leads to the Productivity Tracking System module, handled by the **productivityTrackingSystem()** function, which allows the employee to record their tasks and assess their productivity. The third option exits the application, closing the program. The **mainmenu()** function is responsible for displaying the menu, capturing the user's input, and directing the program flow based on the selected option.

*1.3.2 Module 2: **Remote Work Readiness Submenu***

The **Remote Work Readiness Submenu Module** provides a navigational interface for users to access different functionalities related to remote work readiness. This module is implemented through the **remoteReadinessSubmenu()** function, which displays a menu with three options: 1) Remote Work Readiness, 2) Generate Safety Report, and 3) Return to Main Menu.

When a user selects the first option, Remote Work Readiness, the **remoteWorkReadiness()** function is triggered, initiating the process of assessing the suitability of their remote work environment. The second option, Generate Safety Report, calls the **generateSafetyReport()** function to create a report based on the data collected from the remote work readiness assessments. Lastly, selecting the third option, Return to Main Menu will exit the submenu and return the user to the primary menu interface.

*1.3.3 Module 3: **Remote Work Readiness***

The **Remote Work Readiness Module** is designed to evaluate the suitability of an employee's home environment for remote work. This module is initiated through the **remoteWorkReadiness()** function, which starts by presenting a series of safety and preparedness questions. The process begins with the **askQuestion()** function, which prompts the employee with questions and requires answers in a Yes/No format. The responses are then validated to ensure they are not empty and conform to expected inputs.

The employee answers these questions, and the system records their responses in a file named according to the entry number, such as **EntryNumber.txt**, managed by **updateEntryNumber()**. This file stores the detailed responses for later review. The **getCurrentEntryNumber()** function is used to retrieve and update the entry number, ensuring that each submission is uniquely identified and tracked.

After answering the questions, the module provides feedback based on the collected responses. If the responses indicate that the workspace is suitable, the system will confirm this; otherwise, it may suggest improvements. The module is designed to systematically gather relevant information, store it accurately, and provide actionable insights to help employees create an effective remote working environment. Managers can review these assessments to ensure that employees meet the necessary criteria for remote work.

*1.3.4 Module 4: **Generate Safety Report***

The **Generate Safety Report Module** is implemented through the **generateSafetyReport()** function and produces a comprehensive report on the safety and suitability of the remote work environment. When triggered, this function retrieves and processes data from the **SafetyChecklist.txt file**, which contains responses to safety-related questions collected during the **remoteWorkReadiness()** assessments. It displays the data in a clear table format, including the total number of 'Yes' and 'No' answers, as well as a summary of the workspace's suitability. Once the report is finalized, users are notified of its successful creation. This module ensures that the safety assessment data is effectively compiled and presented so that the employees and manager can evaluate the remote work environments.

*1.3.5 Module 5: **Productivity Tracking System***

The **Productivity Tracking System Module**, managed by the **productivityTrackingSystem()** function, facilitates the management and analysis of work productivity data. This function displays three options: 1) Log Work Activity & Track Productivity Metrics 2) Generate Report, and 3) Return to Main Menu. When an employee selects option 1, the system calls the **workProductivityLog()** function, which prompts the employee to enter details about their work activities and categorize each task as productive or non-productive. This data is saved in the **WorkLog.txt file**. After logging activities, the employee can choose option 2, which triggers the **generateProductivityReport()** function. This function calculates and displays productivity metrics, providing a summary of the number of productive and non-productive tasks based on the logged data, essential for performance evaluation. Additionally, this option generates a detailed report consolidating information from **WorkLog.txt**, including logged activities, their categorizations, and a summary of productivity metrics, offering insights into work patterns and performance. If the employee selects option 3, the **mainmenu()** function is called, allowing the employee to navigate back to the main menu to access other functionalities of the system. The **productivityTrackingSystem()** function ensures that activities are logged before tracking metrics and generating reports.

*1.3.6 Module 6: **Generate Productivity Report***

The **Generate Productivity Report Module**, managed by the **generateProductivityReport()** function, is designed to produce a comprehensive report on work productivity based on the data collected. When this function is called, it retrieves and processes the data stored in the **WorkLog.txt file**, which includes records of all logged work activities and their categorizations as either productive or non-productive. The function then calculates key productivity metrics, including the total number of productive and non-productive tasks. It generates a detailed report that presents this information in a clear and structured format. This report includes the activity log, detailing each task with information on its name, start and end times, total duration, status, and additional notes. Further details include the activity's category, focus level, distractions encountered, productivity status, qualitative evaluation,

satisfaction level, and feedback. The report serves as an essential tool for performance evaluation, helping employees and managers to analyze productivity. Once the report is generated, the **generateProductivityReport()** function ensures that users are notified of its successful creation, enabling them to review the findings.

*1.3.7 Module 7: **File Handling and Data Storage***

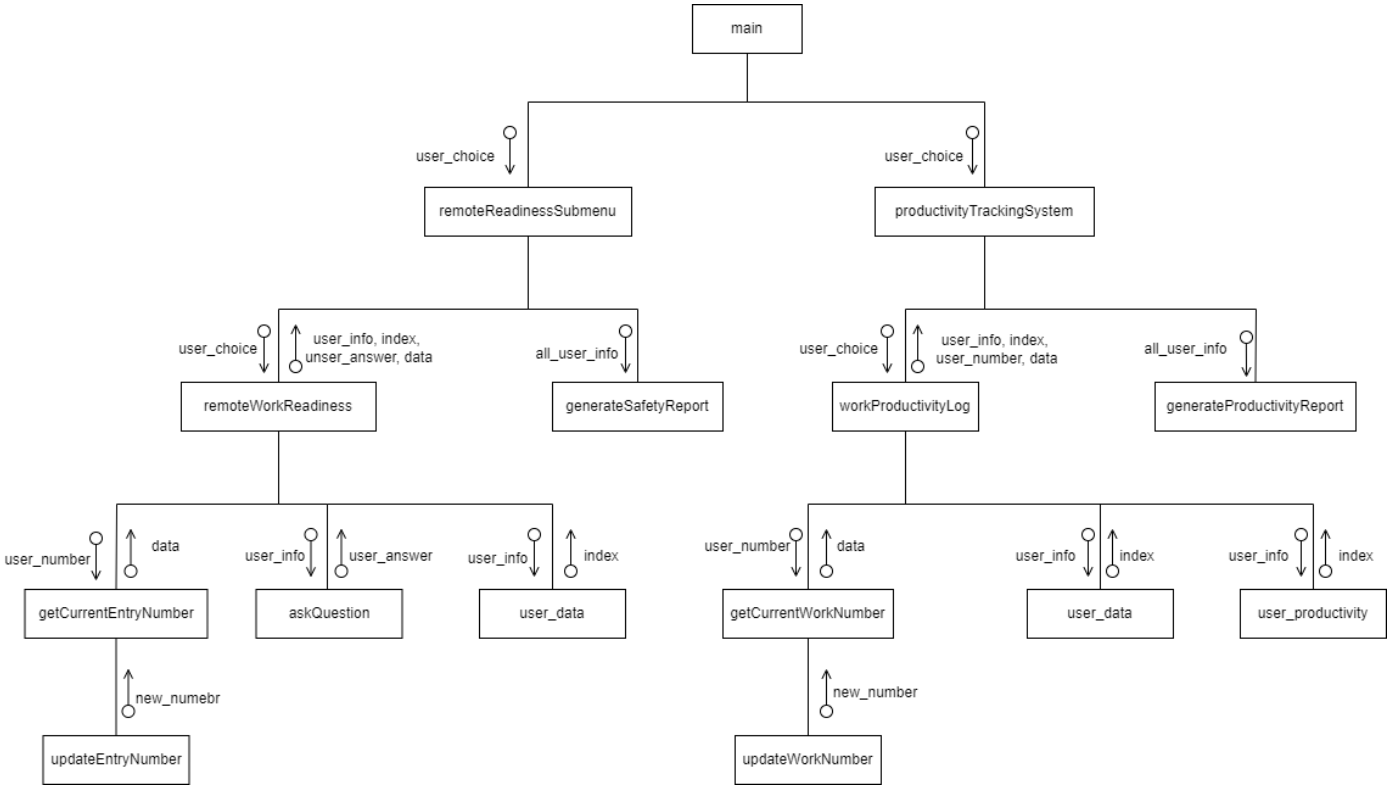
For Employees:

The File Handling and Data Storage Module ensures the seamless and secure management of employee data. When employees provide responses to questions or enter productivity ratings, the system automatically saves this information to the appropriate text files. Specifically, functions such as **updateEntryNumber()** and **getCurrentEntryNumber()** handle the storage and retrieval of remote work readiness data, while **updateLoginNumber()** and **getCurrentLoginNumber()** manage the data related to productivity tracking. This continuous logging process maintains a reliable record of all employee activities, ensuring that data is consistently updated and accessible.

For Managers:

Managers utilize this module to access and analyze all data stored in the text files. This functionality allows them to retrieve and review detailed records of employee responses, productivity metrics, and other relevant information. By examining these files, managers can track long-term trends and performance patterns in remote work environments and employee productivity.

1.4 Structure Chart



1.5 Pseudocode

1）Int main() FUNCTION

```
Start
  mainmenu()
   if choice == 1 then
    remoteReadinessSubmenu()

   else if choice == 2 then
    productivityTrackingSystem()
      end if

  if choice != 0 then
    Display "Exiting the program. Goodbye!"
  end if
Stop
```

2）int mainmenu() FUNCTION

Start

DISPLAY "Welcome to the Remote Work Readiness and Productivity Tracking System"

DISPLAY "Remote Work Readiness Menu [1]"

DISPLAY "Productivity Tracking System Menu [2]"

DISPLAY "Exit [0]"

DISPLAY "Please select an option (1, 2, or 0): "

INPUT choice

WHILE (input not a valid integer) OR (choice is less than 0) OR (choice is greater than 2)

CLEAR input

IGNORE any remaining input in the buffer

DISPLAY "Invalid input. Please enter a number (1, 2, or 0): "

INPUT choice

RETURN choice

END

3) void remoteReadinessSubmenu() FUNCTION

```
BEGIN FUNCTION remoteReadinessSubmenu
  DECLARE INTEGER subChoice
  DECLARE BOOLEAN SafetyChecklistCompleted = FALSE

  DO
    DISPLAY "Remote Work Readiness Menu selected. [1]"
    DISPLAY "Remote Work Readiness Menu"
    DISPLAY "Remote Work Readiness safety Checklist --[1]"
    DISPLAY "Generate Report ------------------------[2]"
    DISPLAY "Return to Main Menu ---------------------[0]"
    DISPLAY "Please select an option (1, 2, or 0): "

    INPUT subChoice

    WHILE subChoice is not a valid number between 0 and 2
      CLEAR input buffer
      DISPLAY "Invalid input. Please enter a number (1, 2, or 0): "
      INPUT subChoice
    END WHILE

    SWITCH subChoice
      CASE 1:
        CLEAR screen
        DISPLAY "Remote Work Readiness selected."
        CALL remoteWorkReadiness()
        SET SafetyChecklistCompleted = TRUE
        CLEAR screen
      BREAK

      CASE 2:
```

```
            CLEAR screen
            IF SafetyChecklistCompleted is TRUE THEN
               DISPLAY "Generating Report from SafetyChecklist.txt..."
               CALL generateSafetyReport()
               CLEAR screen
            ELSE
               DISPLAY "Please complete Safety Checklist (option[1]) first, before Generating Report"
               DISPLAY "Enter any key to return..."
               WAIT for user input
               CLEAR screen
            END IF
         BREAK

         CASE 0:
            DISPLAY "Returning to Main Menu..."
         BREAK
      END SWITCH
   WHILE subChoice is not equal to 0
END
```

4) void remoteWorkReadiness() FUNCTION

Define an array of checklist questions

Declare variables:

employeeName

date

managerName

remoteWorkAddress

workAreaDescription

Set entryNumberFileName to "EntryNumber.txt"

Get the current entry number from the file using getCurrentEntryNumber()

Increment entryNumber

Display the entry number header

Do the following until employeeName is not empty:

Prompt "Employee" and read employeeName

If employeeName equals "0", return (exit function)

Do the following until date is not empty:

Prompt "Date" and read date

If date equals "0", return (exit function)

Do the following until managerName is not empty:

Prompt "Manager" and read managerName

If managerName equals "0", return (exit function)

Do the following until remoteWorkAddress is not empty:

Prompt "Remote work site address" and read remoteWorkAddress

If remoteWorkAddress equals "0", return (exit function)

Do the following until workAreaDescription is not empty:

Prompt "Description of work area" and read workAreaDescription

If workAreaDescription equals "0", return (exit function)

Open a file "SafetyChecklist.txt" in append mode (outFile)

Write entry number header to outFile

Write user information (employeeName, date, managerName, remoteWorkAddress, workAreaDescription) to outFile

Iterate through the array of questions:

For each question, call askQuestion() with the question and outFile

Write the answer to the outFile with a blank line

Close outFile after writing all data

Update the entry number in "EntryNumber.txt" using updateEntryNumber()

Display "Entry completed and saved to SafetyChecklist.txt"

Wait for the user to continue

End

5） void askQuestion(string question, ofstream* outFile, int* countY, int* countN) FUNCTION*

Declare variable answer to store the user's answer

Display the question with "(Y-Yes/n-No):" prompt

Read user's answer into answer variable

While answer is not valid (use isValidAnswer function):

Display "Invalid answer. Please enter (Y-Yes or N-No) [case insensitive]:"

Read user's answer again

Write the question and the answer (with brackets) to outFile, followed by a newline

Display a blank line

End Function

6）void generateSafetyReport() FUNCTION

Open the file "SafetyChecklist.txt" in read mode (in_File)

If the file is successfully opened:

Display "Safety Checklist Report:"

While there are lines to read from the file:

Read the current line from the file

Display the line

Close the file after reading

Else (if the file could not be opened):

Display "Unable to open the SafetyChecklist.txt file."

Display "Possible Reason 1: No entry for Safety Checklist (option[1]). Please complete it at least once."

Display "Possible Reason 2: SafetyChecklist.txt file has not been created."

Wait for the user to continue

Clear the screen

End Function

7）int getCurrentEntryNumber() FUNCTION

Open the file "EntryNumber.txt" in read mode (inFile)

Initialize entryNumber to 0

If the file is successfully opened:

Read the entry number from the file into entryNumber

Close the file after reading

Return entryNumber

End Function

8）void updateEntryNumber(int newEntryNumber) FUNCTION

Open the file "EntryNumber.txt" in write mode (outFile)

If the file is successfully opened:

Write newEntryNumber to the file

Close the file after writing

Else (if the file could not be opened):

Display "Unable to update entry number file."

Wait for the user to continue

End Function

9）bool isValidAnswer(string& answer) FUNCTION

If answer is equal to "Yes", "YES", "yes", "y", "Y", "No", "NO", "no", "n", or "N":

Return true

Else:

Return false

End Function

10）void productivityTrackingSystem() FUNCTION

```
Declare variable subChoice as integer
Declare variable case1CompletedFirst as boolean, initialize to false

Do
   Display "Productivity Tracking System Menu selected. [2]"
   Display "Productivity Tracking System Menu"
   Display "----------------------------------------------------------------"
   Display "Log Work Activity & Track Productivity Metrics ----------[1]"
   Display "Generate Report ------------------------------------------[2]"
   Display "Return to Main Menu --------------------------------------[0]"
   Display "----------------------------------------------------------------"
   Display "Please select an option (1, 2, or 0):"

   While user input for subChoice is invalid or not in range 0-2:
      Clear invalid input
      Ignore invalid characters
      Display "Invalid input. Please enter a number (1, 2, or 0):"

   Switch on subChoice
      Case 1:
         Clear screen
         Call workLog() function
         Set case1CompletedFirst to true

      Case 2:
         If case1CompletedFirst is true:
            Clear screen
            Display "Report List"
            Call generateReport() function
         Else:
```

```
            Clear screen
            Display "Possible Reason 1: No Entry Recorded"
            Display "Please Log your Work Activity and Complete Track Productivity Metrics (Option [1])"
            Display "Before Generating Report"
            Wait for user to continue
            Clear screen
        Case 0:
          Display "Returning to Main Menu..."
    While subChoice is not equal to 0

End Function
```

11）void workProductivityLog() FUNCTION

```
Start

  Open "WorkLog.txt" as workLogFile in append mode

  Declare string name, contact, date, department, supervisor, experience
  Declare int numActivities

  Set workNumber = getCurrentWorkNumber() + 1
  Call updateWorkNumber(workNumber)

  Display "WORK LOG - ENTRY" and workNumber

  // Collect basic information with input validation
  Do
    Display  "Enter Name"
    Input name
  While name is empty

  Do
    Display "Enter Contact No"
    Input contact
  While contact is empty

  Do
    Display "Enter Today's Date"
    Input date
  While date is empty

  Do
    Display "Enter Department"
```

```
    Input department
While department is empty

Do
    Display "Enter Supervisor"
    Input supervisor
While supervisor is empty

While true
    Display "Enter number of activities"
    Input numActivities
    If numActivities is valid positive integer
        Break
    Else
        Display "Invalid input, enter a valid number"

If workLogFile is not open
    Display error message and exit

Write workNumber, name, contact, date, department, supervisor to file

Set totalschTimeMinutes = 0

Declare string array activities of size numActivities

For i from 0 to numActivities - 1
    Declare string activity, startTime, endTime, status, notes
    Declare int totalMinutes

    Display "Enter Activity i+1"
    Input activity
    Store activity in activities[i]
```

```
    Do
        Display "Enter Start Time (HH:MM)"
        Input startTime
    While startTime is not valid

    Do
        Display "Enter End Time (HH:MM)"
        Input endTime
    While endTime is not valid

    Display "Enter Status"
    Input status
    Display "Enter Notes"
    Input notes

    Set startMinutes = convert startTime to minutes
    Set endMinutes = convert endTime to minutes
    Set totalMinutes = endMinutes - startMinutes
    If totalMinutes < 0
        Adjust totalMinutes for time after midnight

    totalschTimeMinutes += totalMinutes

    Write activity, startTime, endTime, totalMinutes, status, notes to file

Write totalschTimeMinutes to file

Write productivity tracking header to file
Initialize productiveTasks and nonProductiveTasks counters

For i from 0 to numActivities - 1
```

```
Declare string category, productive, evaluate, satisfaction, feedback
Declare int focusLevel, distractions

Display "Enter Category for activities[i]"
Input category

Display "Rate Focus Level (1-5)"
Input focusLevel

Display "Rate Distractions Level (1-5)"
Input distractions

Do
   Display "Was the activity productive? (Yes/No)"
   Input productive
While productive is not valid

Display "Enter Evaluation"
Input evaluate

Display "Enter Satisfaction"
Input satisfaction

Display "Enter Feedback"
Input feedback

If productive is Yes
   Increment productiveTasks
Else
   Increment nonProductiveTasks

Write activity, category, focusLevel, distractions, productive, evaluate, satisfaction, feedback to file
```

```
    Display "Enter your overall experience"
    Input experience
    Write experience to file

    Write productiveTasks, nonProductiveTasks, numActivities to file

    Delete activities array

    Close workLogFile

    Display "Work log saved successfully"
    Wait for user input

End
```

12）bool isValidTime(const string& timeStr) FUNCTION

```
IF length of timeStr is not 5 OR the third character is not ':'
    RETURN false // Invalid format

SET hours to integer value of the substring of timeStr from index 0 to 2 // Get hours (first two characters)
SET minutes to integer value of the substring of timeStr from index 3 to 5 // Get minutes (last two characters)

IF hours is within the range [0, 23] AND minutes is within the range [0, 59]
    RETURN true // Valid time format
ELSE
    RETURN false // Invalid time format
```

13）int timeToMinutes(const string& timeStr) FUNCTION

```
DECLARE hours AS INTEGER
DECLARE minutes AS INTEGER
DECLARE colon AS CHAR

CREATE timeStream AS istringstream(timeStr)

timeStream >> hours >> colon >> minutes

RETURN (hours * 60) + minutes
```

14）string timeToMinutes(const string& timeStr) FUNCTION

```
DECLARE hours AS INTEGER
hours = totalMinutes / 60

DECLARE minutes AS INTEGER
minutes = totalMinutes % 60

DECLARE timeStream AS ostringstream

timeStream << setw(2) << setfill('0') << hours << ":"
        << setw(2) << setfill('0') << minutes

RETURN timeStream.str()
```

15）int getCurrentWorkNumber() FUNCTION

```
DECLARE workNumber AS INTEGER
workNumber = 0

DECLARE workFile AS ifstream
workFile.open("WorkNumber.txt")

IF workFile.is_open() THEN
   // Step 4: Read the work number from the file
   workFile >> workNumber

   workFile.close()

ELSE

   DISPLAY "Error: Unable to open WorkNumber.txt. Starting from entry 1."
END IF

RETURN workNumber
```

16）void updateWorkNumber(int newWorkNumber) FUNCTION

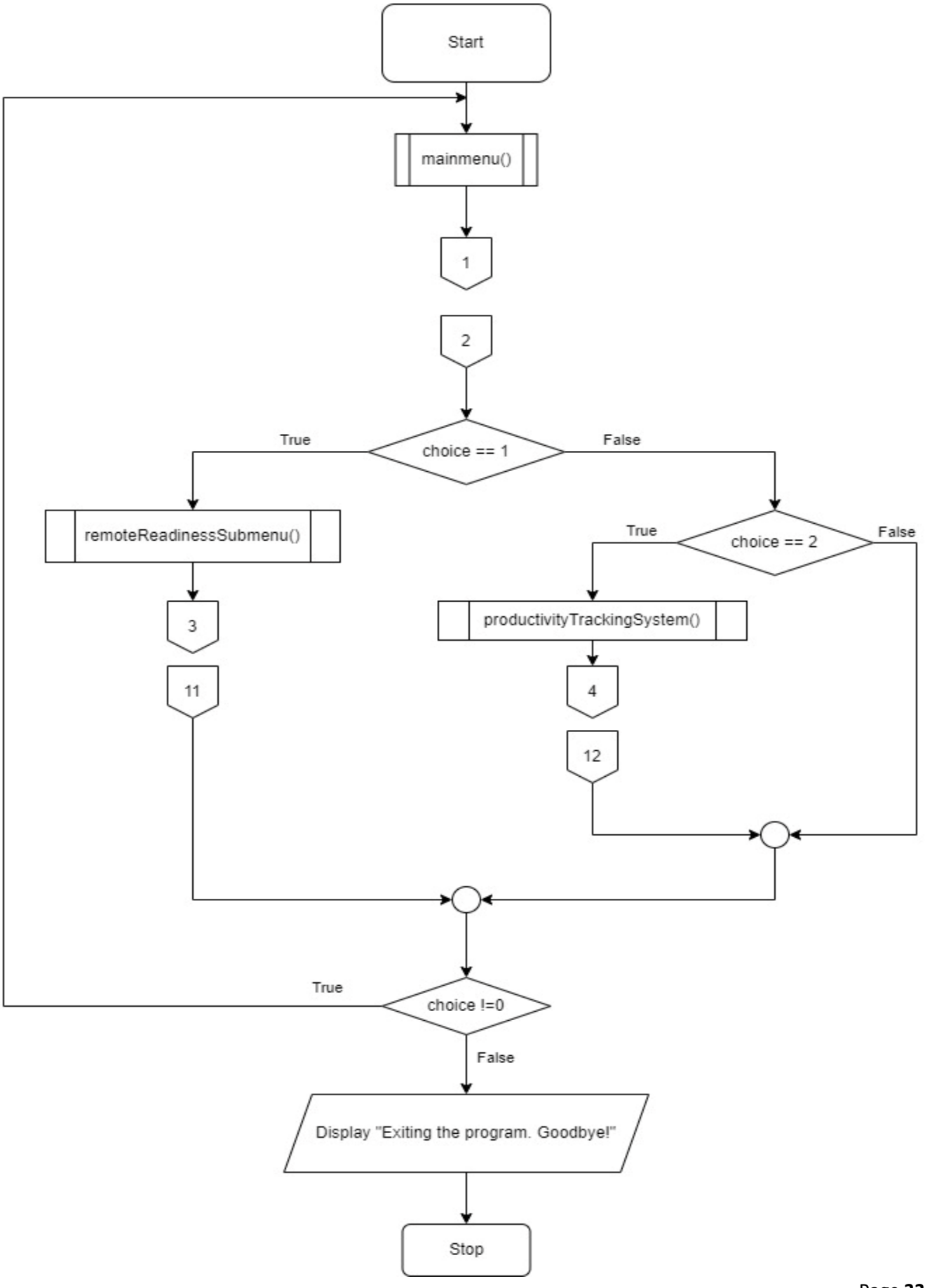
```
    DECLARE workFile AS ofstream
    workFile.open("WorkNumber.txt")

    IF workFile.is_open() THEN
        workFile << newWorkNumber

        workFile.close()
    ELSE
        DISPLAY "Error: Unable to update WorkNumber.txt."

        DISPLAY "Enter any key to continue..."
        CALL cin.ignore()
        CALL cin.get()
    END IF
END FUNCTION
```

17）void generateProductivityReport() FUNCTION

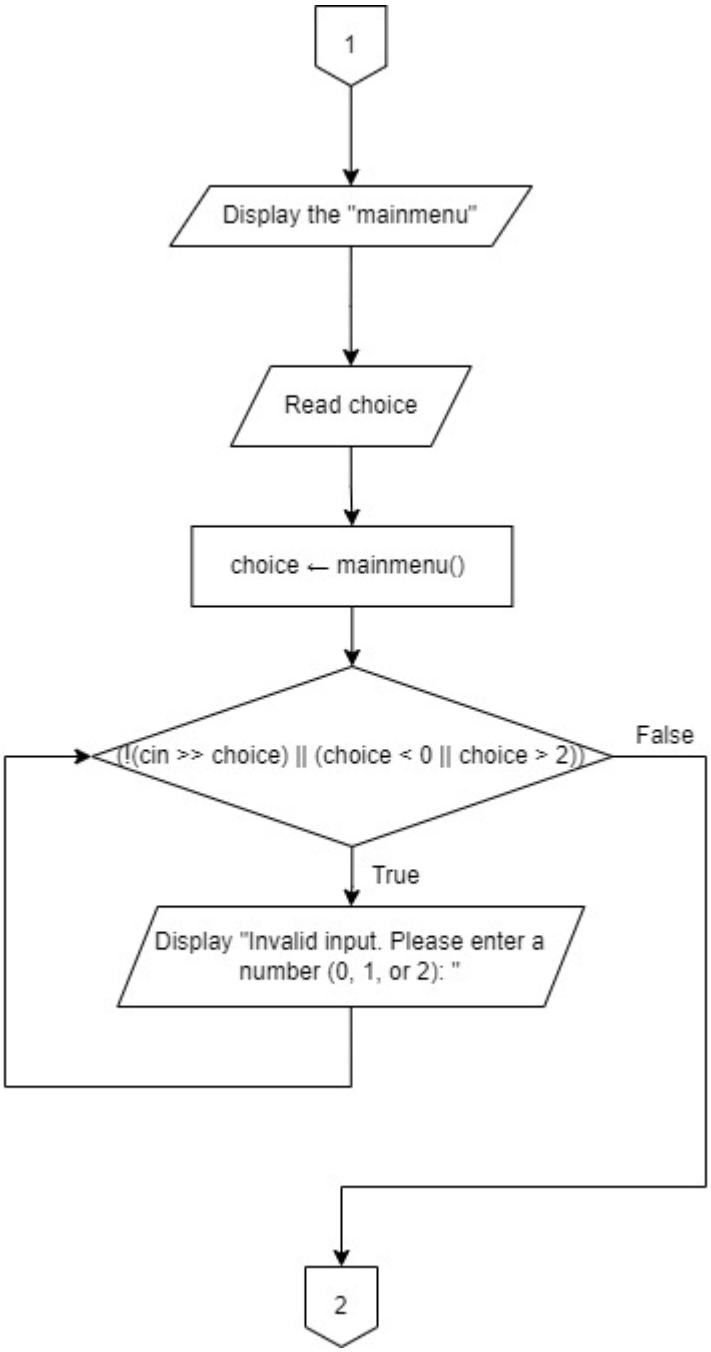
```
  DECLARE in_File AS ifstream
  in_File.open("WorkLog.txt")

  IF in_File.is_open() THEN
    DISPLAY "Generated Report:"

    DECLARE line AS STRING
    WHILE getline(in_File, line) DO
      DISPLAY line
    END WHILE

    in_File.close()

  ELSE
    DISPLAY "Unable to open the Work Log file."
  END IF

  DISPLAY "Enter any key to continue..."
  CALL cin.ignore()
  CALL cin.get()

  CALL system("cls")
END FUNCTION
```

1.6 Flowchart

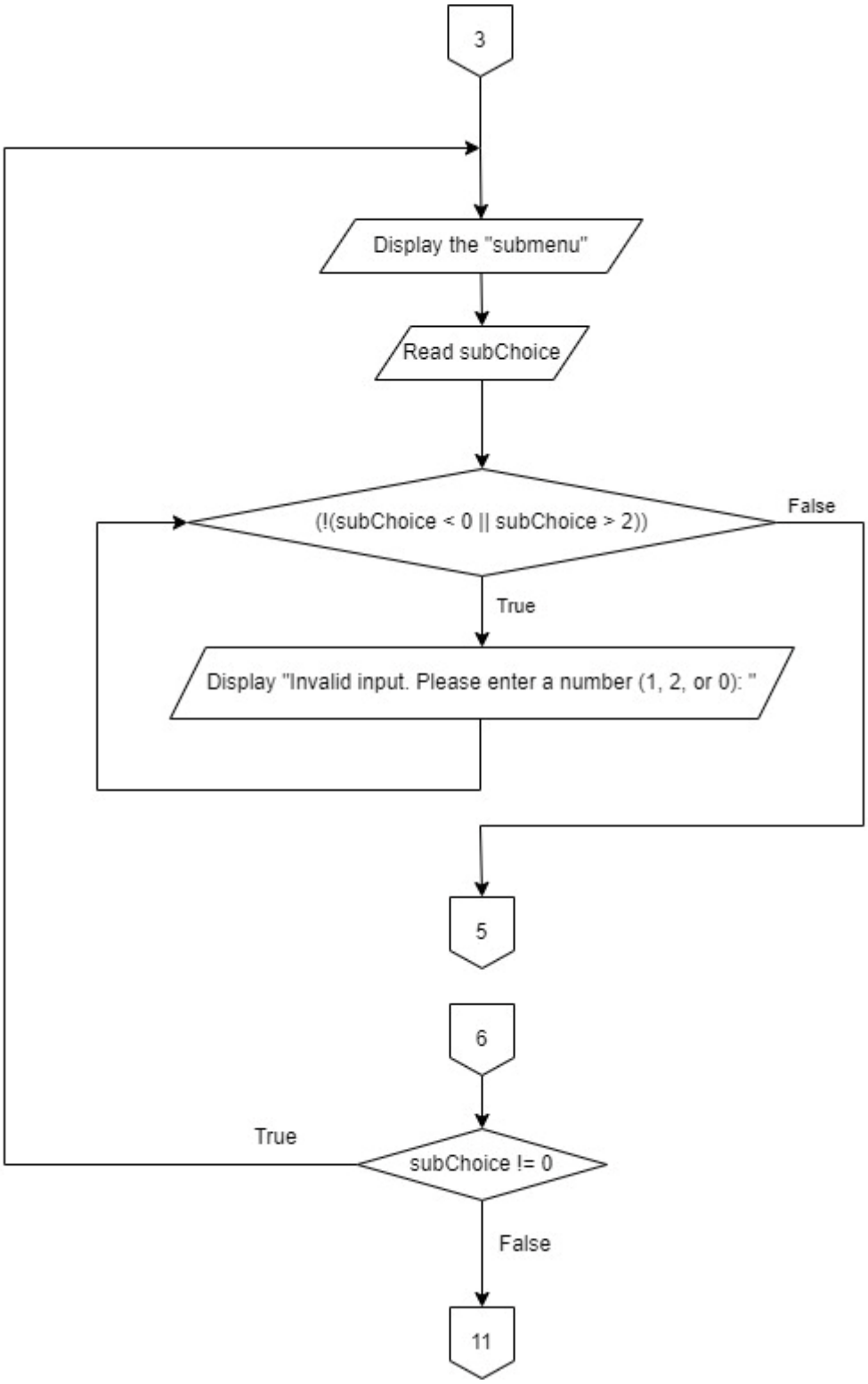
1.6.1 mainmenu



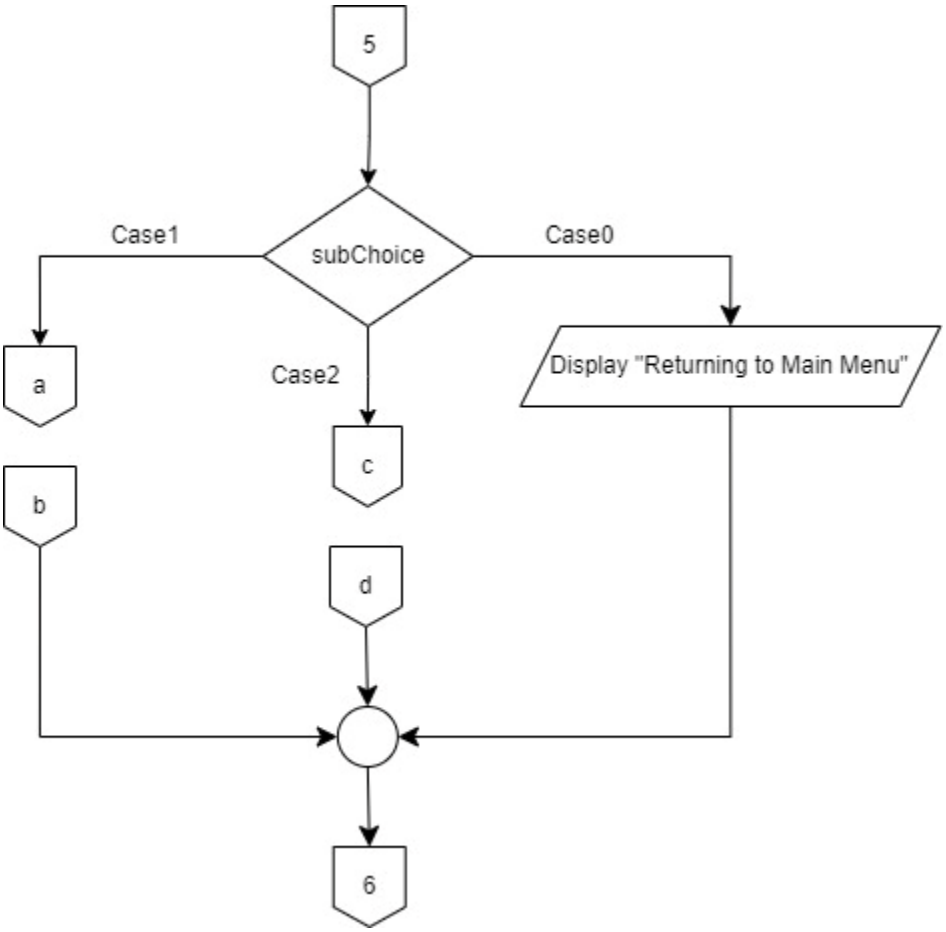
1.6.2 mainmenu()



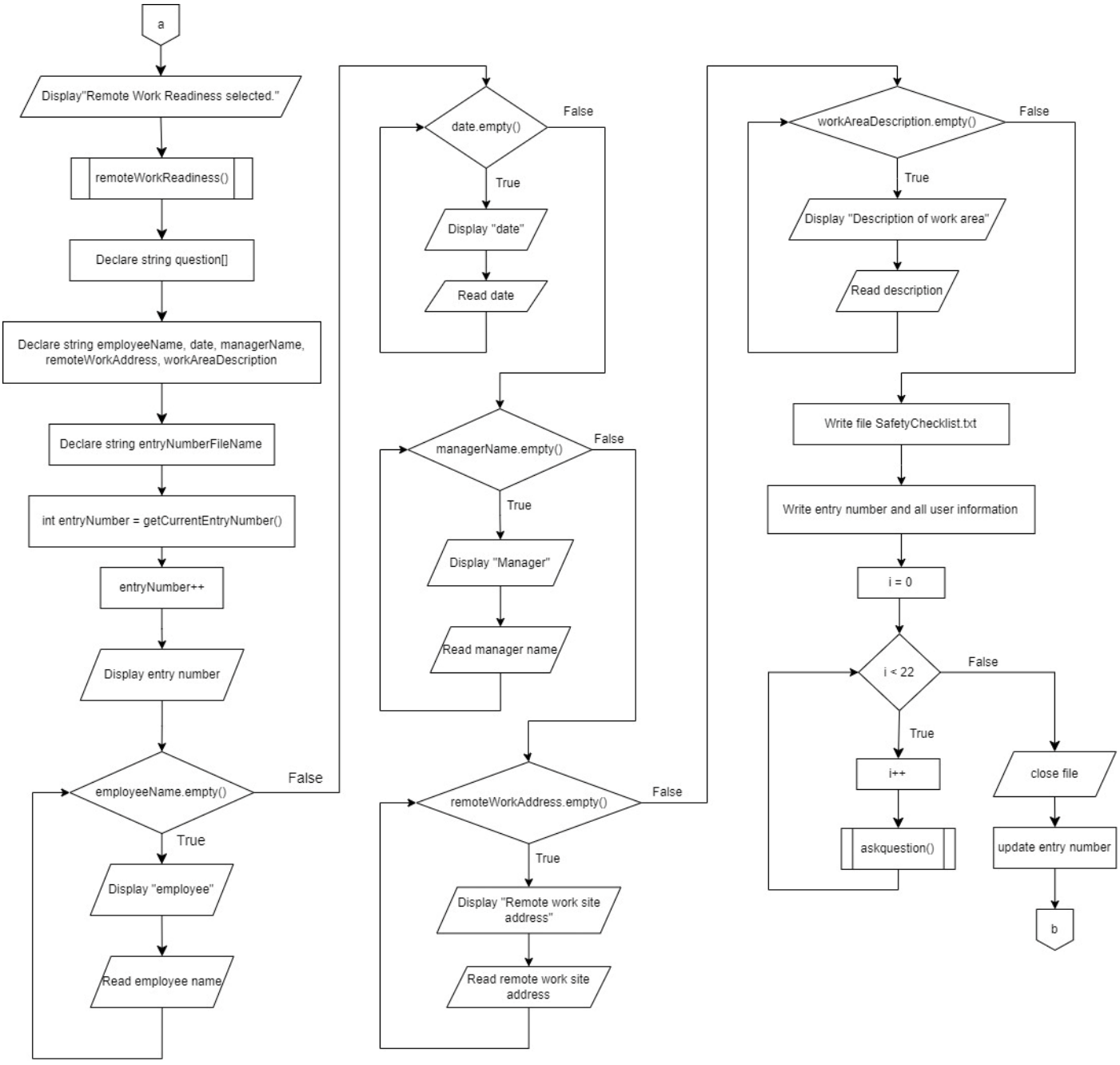
1.6.3 remoteReadiness



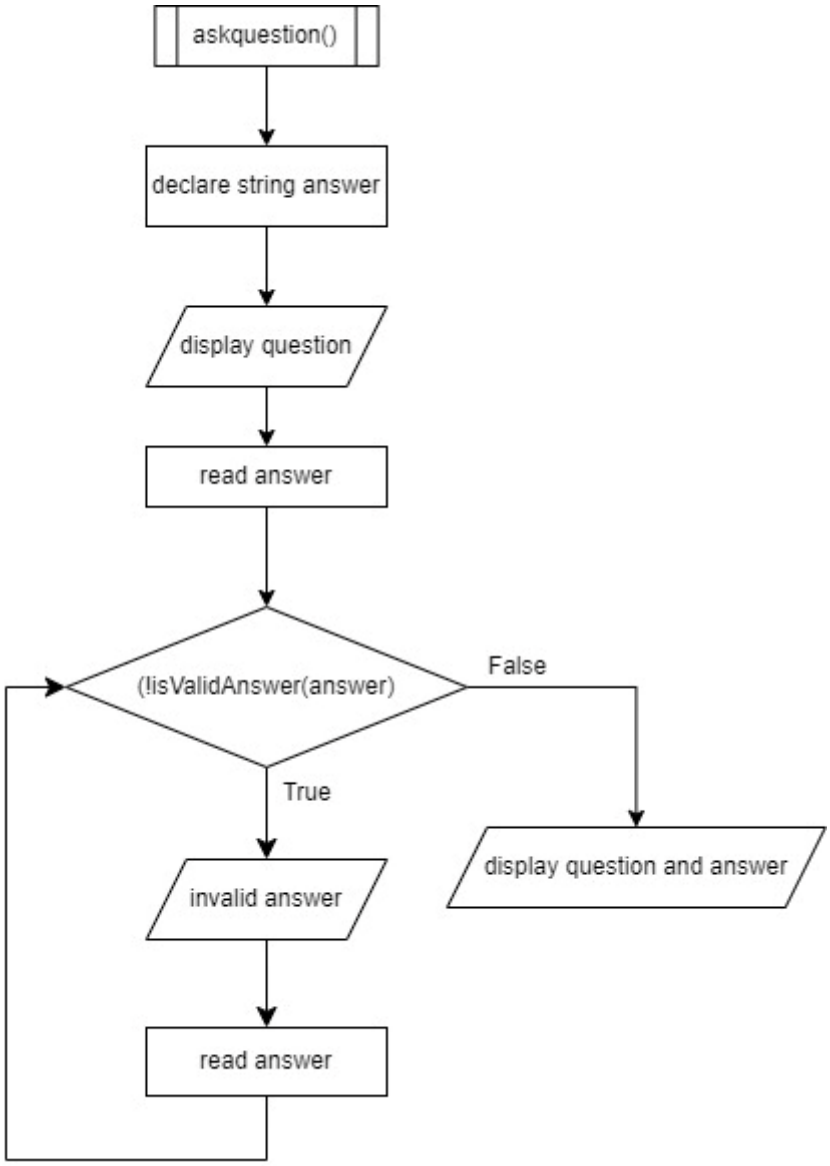
1.6.4 switch(subchoice)



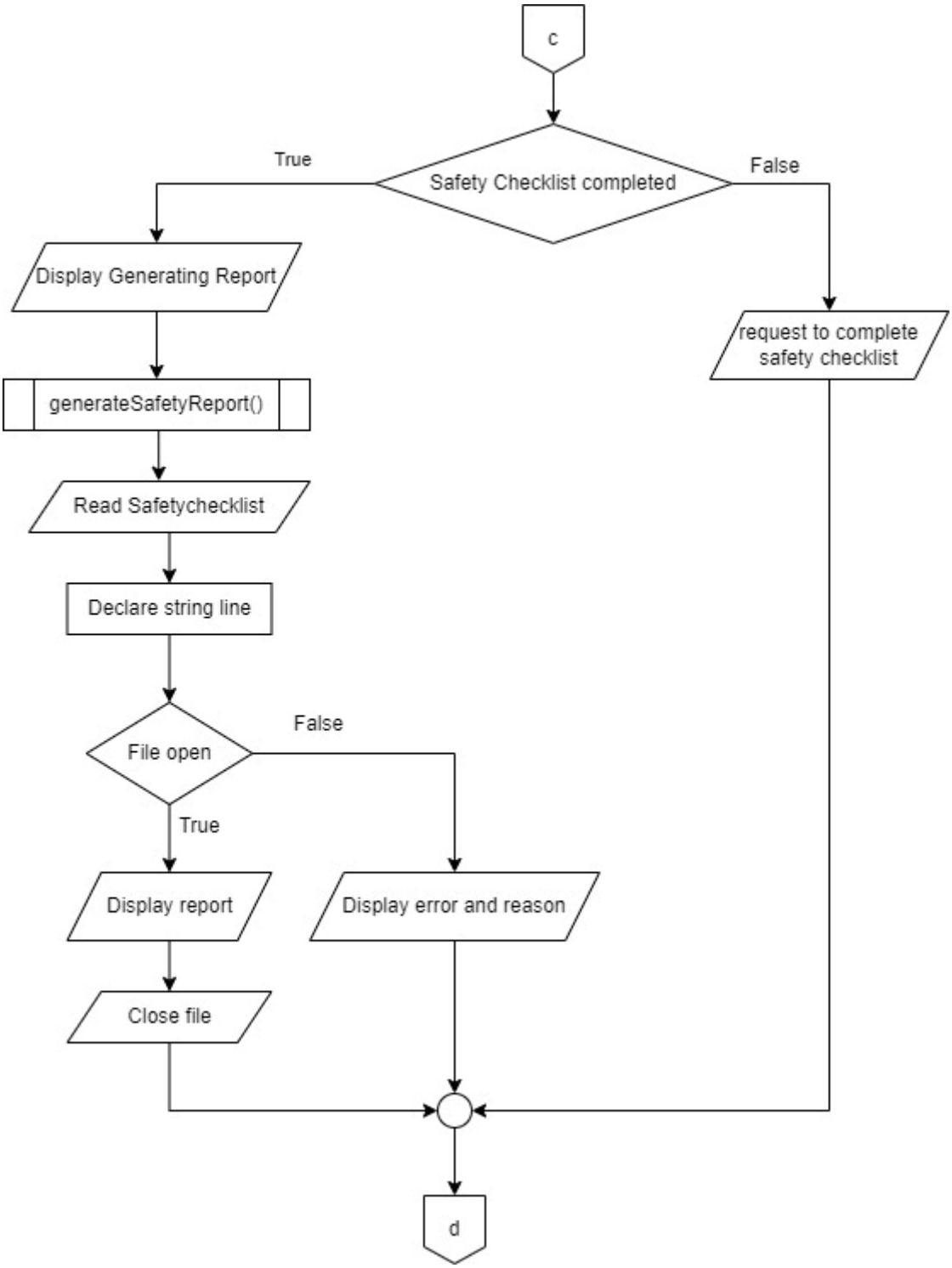
1.6.5 subchoice(case1)



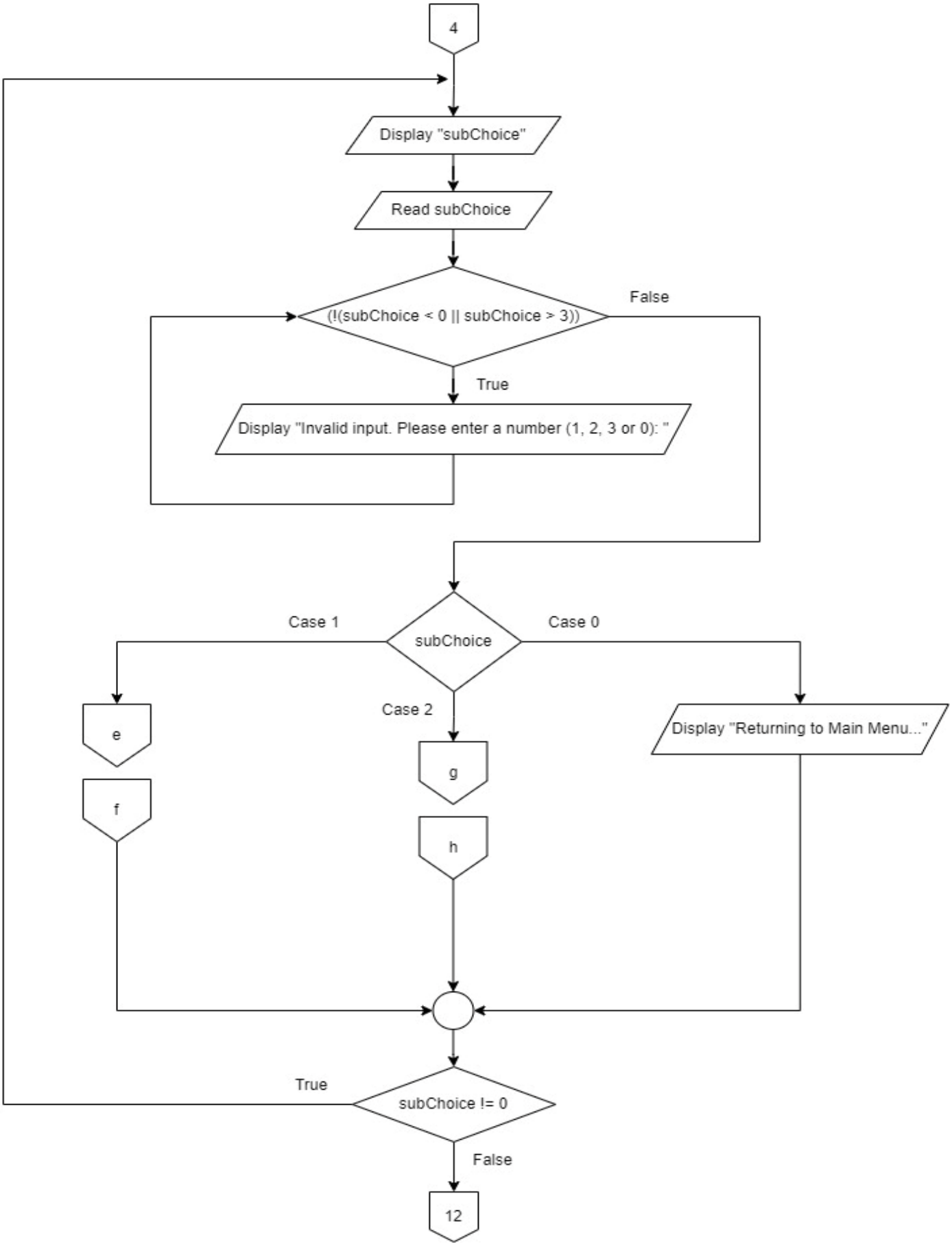
1.6.6 subchoice(case1.1)



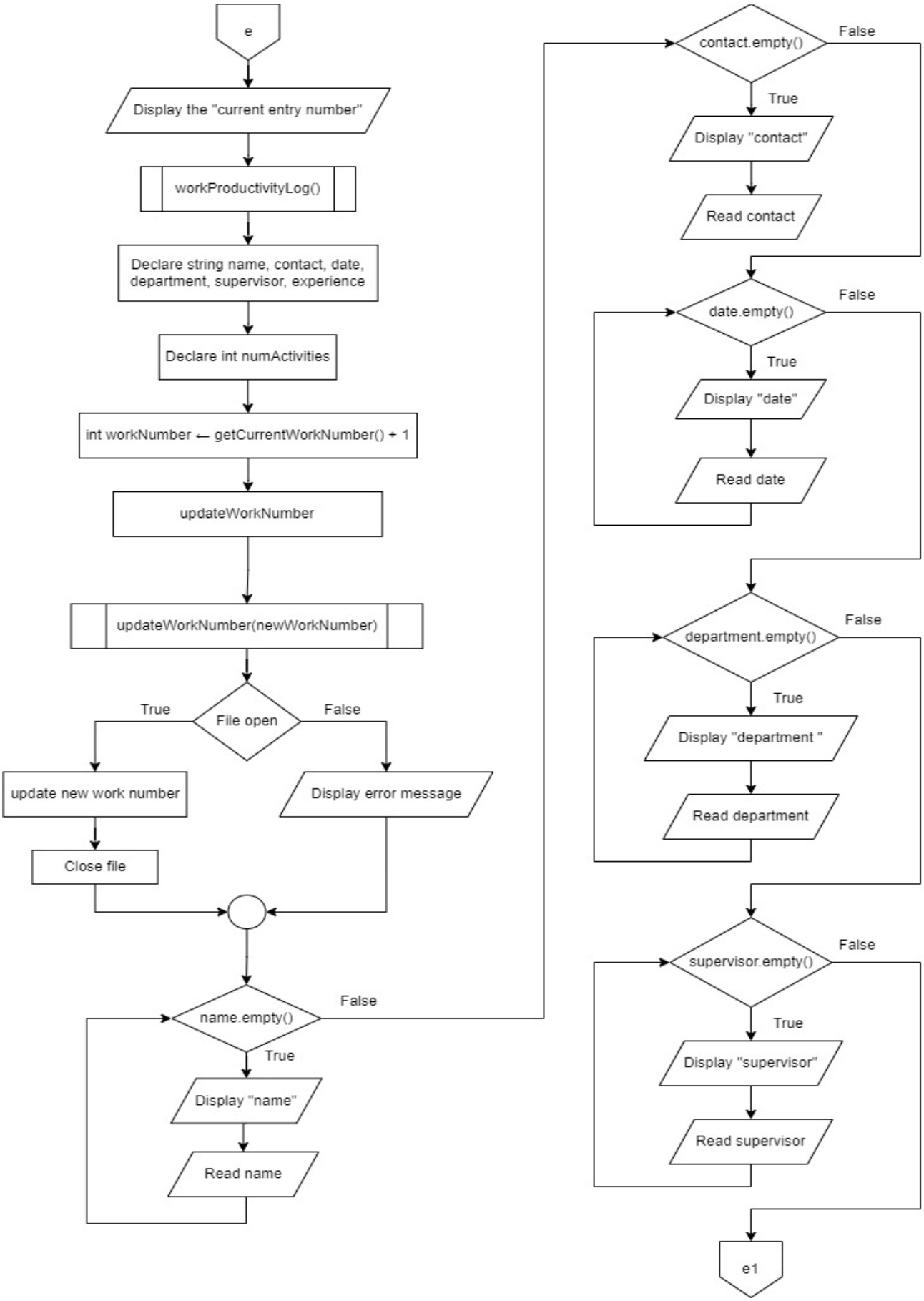
1.6.7 subchoice(case2)



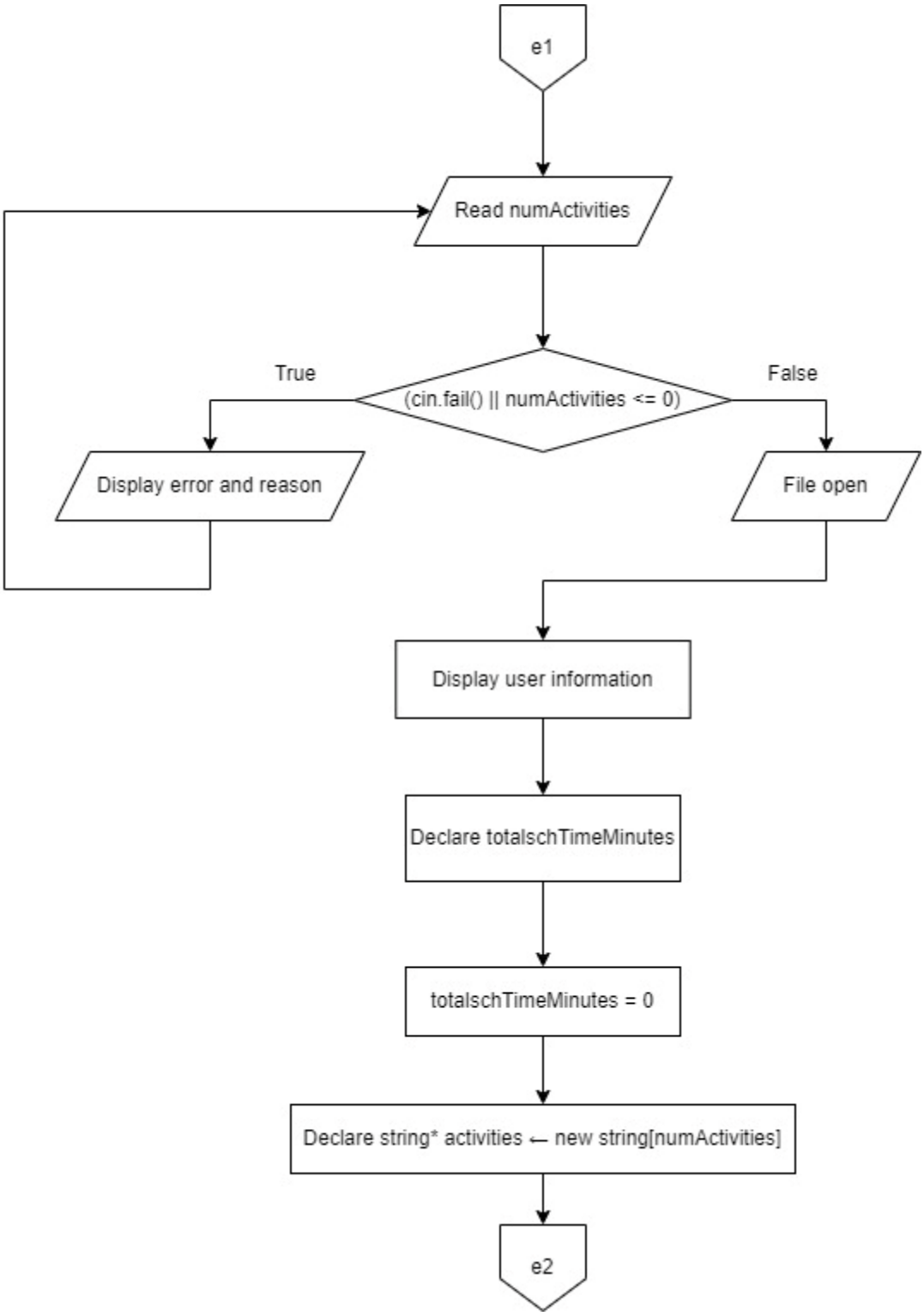
1.6.8 productivityTracking



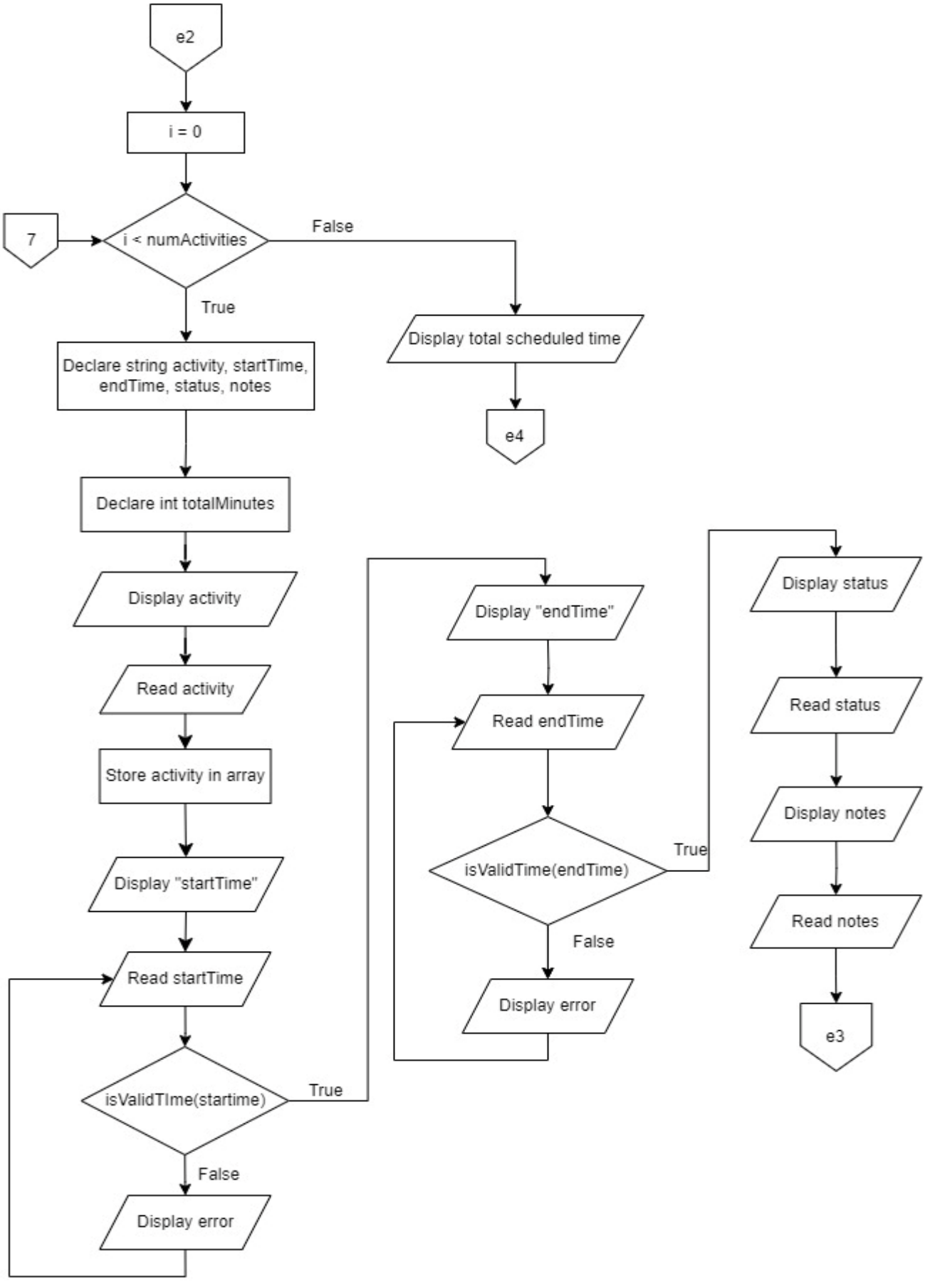
1.6.9 subChoice(case1)



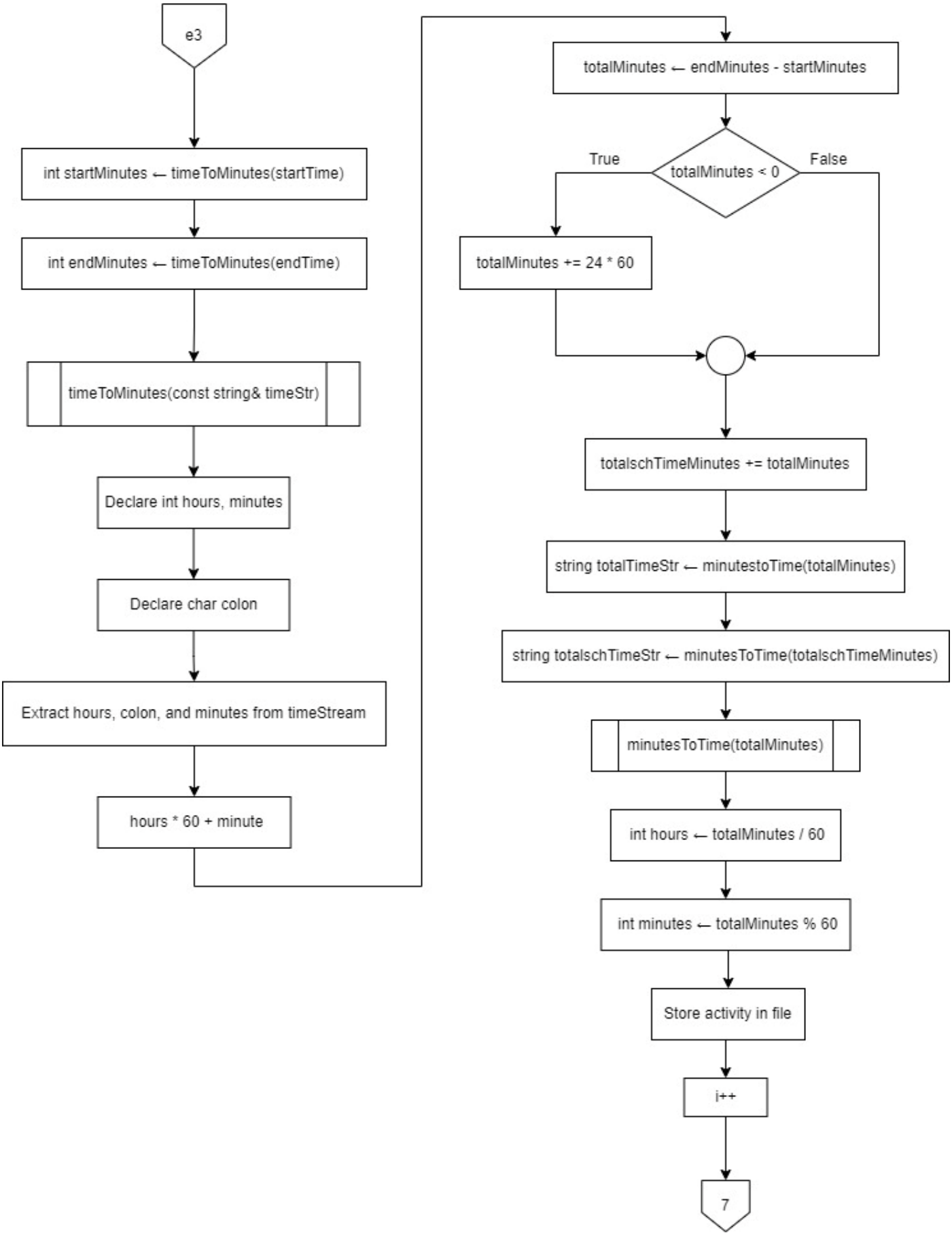
1.6.10 subChoice(case1.1)



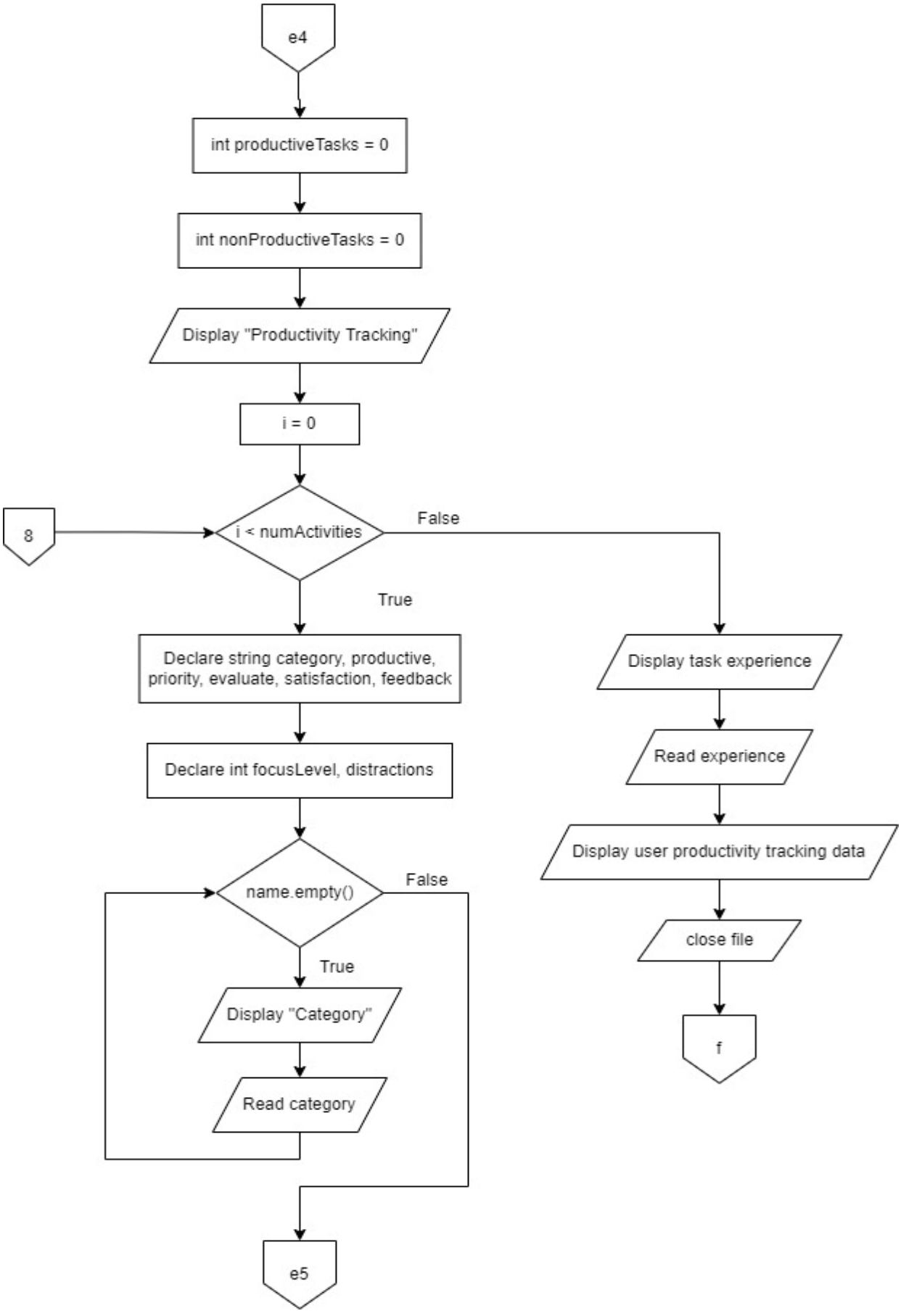
1.6.11 subChoice(case1.2)



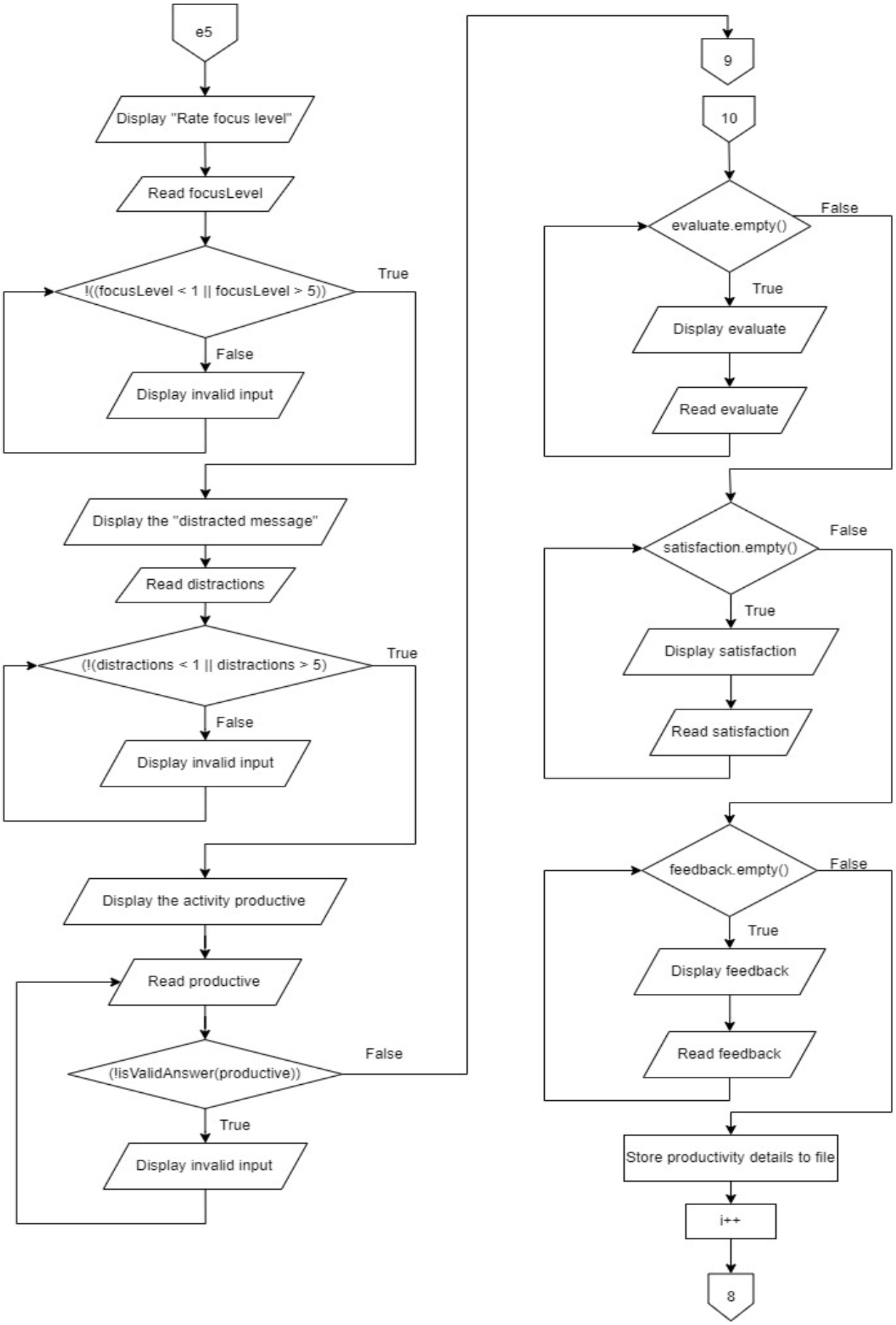
1.6.12 subChoice(case1.3)



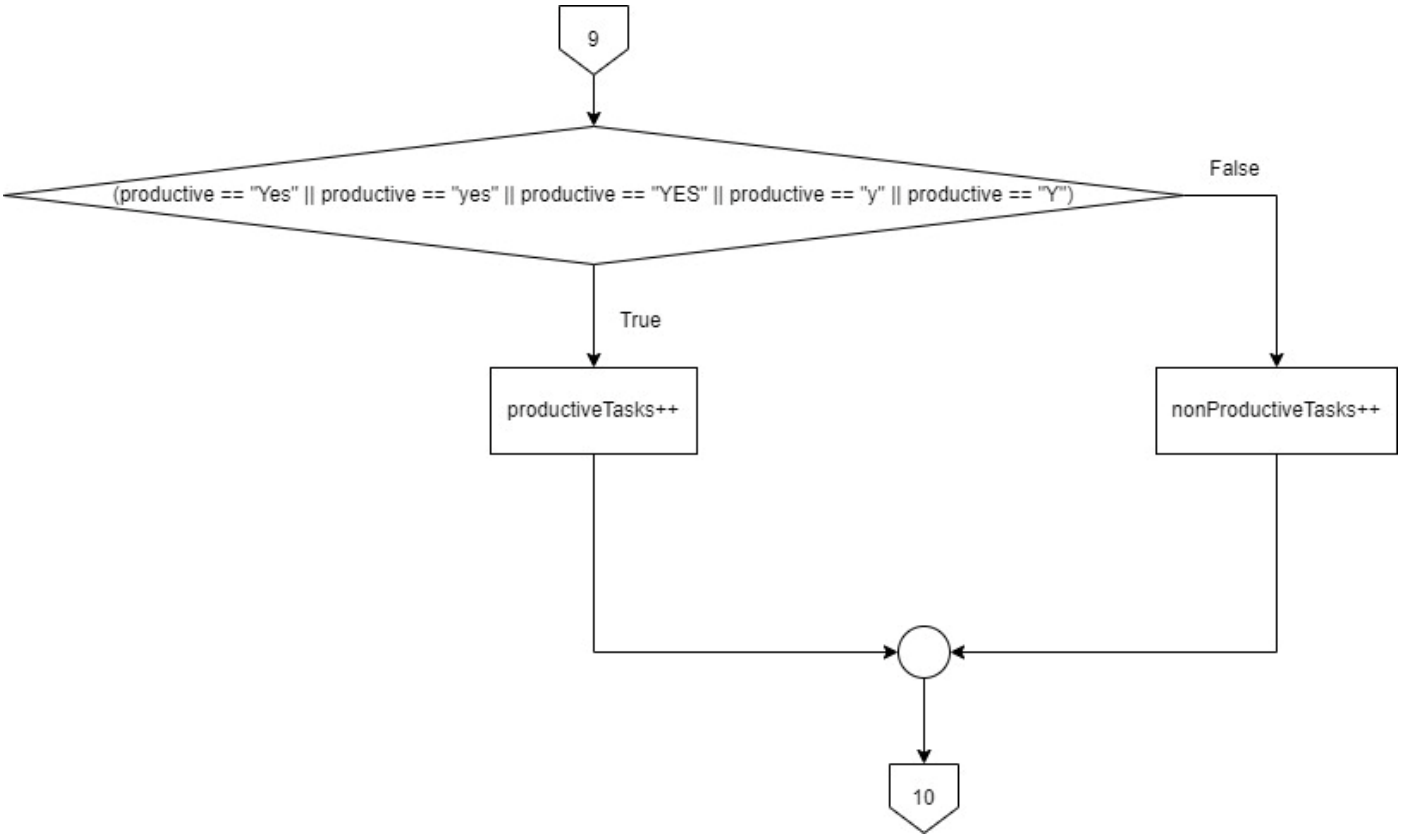
1.6.13 subChoice(case1.4)



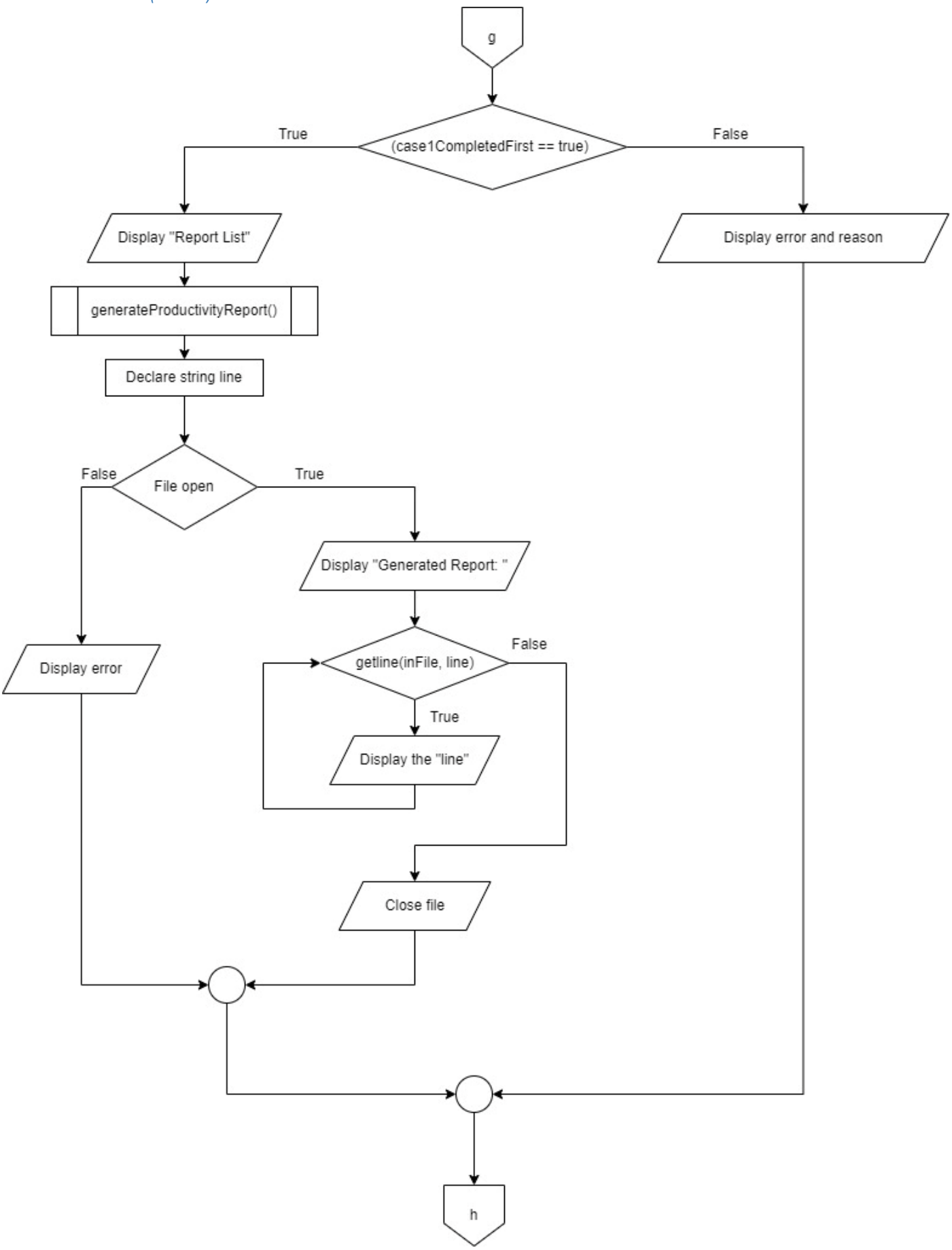
1.6.14 subChoice(case1.5)



1.6.15 subChoice(case1.5.1)



1.6.16 subChoice(case2)



1.7 Special Issues/Constraints

Time constraints

Limited Testing Time: There might not be enough time for comprehensive testing due to the tight deadlines, this causes some errors and bugs that we don’t have enough time to fix.

Quick Decision-Making: We had to decide to focus on the project's main goals rather than the complex and essential features. This can let us work more efficiently to allocate our resources and ensure that our final product is delivered on time and meets the necessary needs.

Maintenance constraints

Code Maintainability: We focused on writing clean and understandable code by providing sufficient comments throughout the program to make it clear and precise. For example, we used consistent and meaningful naming for the variables and functions, ensuring that the code was easier to maintain and understand for the developers.

Lack of Testing and Debugging Code: Without sufficient testing and debugging code, we found it was difficult and challenging to maintain the entire program. We should implement automated tests that run each row of the code and make changes so it can resolve the issues immediately.

1.8 Reference

1) How to do proper documentation

Andreoletti.D . *et al.* (2012) *How to document a method with parameter(s)?*, *Stack Overflow*. Available at: https://stackoverflow.com/questions/9195455/how-to-document-a-method-with-parameters (Accessed: 31 August 2024).

2) How to handle the wrong data type input

ZikZik. *et al.* (2012) *How to handle wrong data type input*, *Stack Overflow*. Available at: https://stackoverflow.com/questions/10349857/how-to-handle-wrong-data-type-input (Accessed: 31 August 2024).

3) How to dynamically allocate memory

PollexBjörn. *et al.* (2011) *C++ dynamically allocated memory*, *Stack Overflow*. Available at: https://stackoverflow.com/questions/8504070/c-dynamically-allocated-memory (Accessed: 31 August 2024).

2 C++ PROGRAM

```
/*
 Program Name: Remote Work Readiness and Productivity Tracking System

 Description:

 This program is designed to assess the readiness of a home office for remote
work and track the productivity of employees.
 It provides two main functionalities:
 1. Remote Work Readiness Checklist: A safety checklist for remote work
environments.
 2. Productivity Tracking System: Allows users to log work activities, track
productivity metrics, and generate reports.

 The program uses a text file to store and retrieve data, including employee
information, safety checklist results,
 and productivity logs. It provides options for generating reports based on the
collected data.
 */

//Preprocessor directives
#include <iostream>
#include <fstream>
#include <string>
#include <cstdlib>
#include <sstream>
#include <iomanip>


using namespace std;

// Function prototypes

// Displays the main menu and handles user input.
int mainmenu();
// Displays the submenu for the Remote Work Readiness section.
void remoteReadinessSubmenu();
// Handles the Remote Work Readiness checklist, including user input and file
operations.
void remoteWorkReadiness();
// Asks a question from the checklist, gets the user's answer, and writes it to
the file.
void askQuestion(string* question, ofstream* outFile, int* countY, int* countN);
// Retrieves the current entry number from the file.
int getCurrentEntryNumber();
// Updates the entry number in the file.
void updateEntryNumber(int newEntryNumber);
// Validates the user's answer to ensure it is either 'Y' or 'N'.
bool isValidAnswer(string& answer);
// Generates a safety report based on the checklist data.
void generateSafetyReport();

// Handles the Productivity Tracking System, including logging work activities
and generating reports.
```

```
void productivityTrackingSystem();
// Logs a work activity for productivity tracking.
void workProductivityLog();
// Validates the time input format (HH:MM).
bool isValidTime(const string& timeStr);
// Converts a time string (HH:MM) to total minutes.
int timeToMinutes(const string& timeStr);
// Converts total minutes to a time string (HH:MM).
string minutesToTime(int totalMinutes);
// Generates a productivity report based on logged work activities.
void generateProductivityReport();
// Retrieves the current work log number from the file.
int getCurrentWorkNumber();
// Updates the work log number in the file.
void updateWorkNumber(int newWorkNumber);


int main() {
    int choice;
    do {
        system("cls"); // Clear screen
        choice = mainmenu(); // Display the main menu and get user choice
        if (choice == 1) {
            cout << endl;
            system("cls");
            remoteReadinessSubmenu(); // Call the function for Remote Work
Readiness Sub menu
        }
        else if (choice == 2) {
            cout << endl;
            system("cls");
            productivityTrackingSystem(); // Call the function for Productivity
Tracking System
        }
    } while (choice != 0); // Continue looping until the user chooses to exit by
entering 0

    cout << "\nExiting the program. Goodbye!" << endl; // Display an exit message

    return 0;
}

/**
* //Function for Main menu

  Displays the main menu for the Remote Work Readiness and Productivity Tracking
System.

  :return int - The user's choice (1 for Remote Work Readiness Menu, 2 for
Productivity Tracking System Menu, 0 for Exit).
 */
int mainmenu() {
    int choice;
```

```
    cout << "Welcome to the Remote Work Readiness and Productivity Tracking
System" << endl;
    cout << "-------------------------------------------------------------------
-\n" << endl;
    cout << "Remote Work Readiness Menu ----------- [1]\n" << endl;
    cout << "Productivity Tracking System Menu ---- [2]\n" << endl;
    cout << "\nExit ------------------------------- [0]\n" << endl;
    cout << "-------------------------------------------------------------------
-" << endl;
    cout << "Please select an option (1, 2, or 0): ";

    // Validate user input for menu choice
    while (!(cin >> choice) || (choice < 0 || choice > 2)) {
        cin.clear(); // Clear the error flag from invalid input
        cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Discard invalid
input
        cout << "\nInvalid input. Please enter a number (1, 2, or 0): ";
    }

    return choice; // Return user's valid choice
}

bool SafetyChecklistCompleted = false; // Used to check if Case 1 was done first
(only for the first time)

/*
  // Function for the Remote Readiness submenu

  Displays the submenu for Remote Work Readiness, allowing the user to select
from
  various options related to work readiness and report generation.
 */
void remoteReadinessSubmenu() {
    int subChoice;

    do {
        cout << "Remote Work Readiness Menu selected. [1]" << endl;
        cout << "\nRemote Work Readiness Menu" << endl;
        cout << "-----------------------------------------\n" << endl;
        cout << "Remote Work Readiness safety Checklist --[1]\n" << endl;
        cout << "Generate Report ------------------------[2]\n" << endl;
        cout << "\nReturn to Main Menu --------------------[0]\n" << endl;
        cout << "-----------------------------------------" << endl;
        cout << "Please select an option (1, 2, or 0): ";

        while (!(cin >> subChoice) || (subChoice < 0 || subChoice > 2)) {
            cin.clear(); // Clear the error flag from invalid input
            cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Discard
invalid input
            cout << "\nInvalid input. Please enter a number (1, 2, or 0): ";
        }

        switch (subChoice) {
        case 1:
            system("cls");
            cout << "Remote Work Readiness selected." << endl;
```

```
            remoteWorkReadiness(); // Call the Remote Work Readiness function
            SafetyChecklistCompleted = true; // Mark checklist as completed
            system("cls");
            break;
        case 2:
            system("cls");
            if (SafetyChecklistCompleted == true) {
                cout << "Generating Report from SafetyChecklist.txt...\n" <<
endl;
                generateSafetyReport(); // Generate the report from
SafetyChecklist.txt
                system("cls");

            }
            else {
                cout << "Please complete Safety Checklist (option[1]) first,
before Generating Report\n" << endl;
                //Wait for user to acknowledge the message and continue
                cout << "\nEnter any key to return...";
                cin.ignore();
                cin.get();
                system("cls");
            }
            break;
        case 0:
            cout << "Returning to Main Menu..." << endl;
            break;
        }
    } while (subChoice != 0);
}



/*
 // Function to handle Remote Work Readiness
  Handles the Remote Work Readiness checklist by asking a series of predefined
questions,
  recording the responses, and generating a report in a text file.

  The function performs the following tasks:
  - Prompts the user for their details and workspace information.
  - Iterates through a list of safety-related questions, asking each question and
recording the answers.
  - Writes the collected data and responses to a file named
"SafetyChecklist.txt".
  - Provides a summary based on the number of 'Yes' and 'No' answers.
  - Updates the entry number for the checklist.
 */

void remoteWorkReadiness() {
    // Define an array of questions for the checklist
    string questions[] = {
        "(1) Is the work space free from excessive noise?",
        "(2) Is adequate lighting (side or rear) provided at the work station?",
```

```
        "(3) Is all electrical equipment free of recognized hazards that could
cause physical harm\n     (frayed wires running through walls, exposed wires
fixed to the ceiling)?                                    ",
        "(4) Is electrical system adequate for office equipment?",
        "(5) Is electrical equipment grounded?",
        "(6) Are surge protectors properly installed?",
        "(7) Are aisles, doorways and floors free of obstructions to permit
visibility and movement?",
        "(8) Is there an exit that allows prompt exiting?",
        "(9) Are phone lines, electrical cords and extension wires secured under
a desk or along a baseboard?",
        "(10) Is the office space neat and clean?",
        "(11) Is a working fire extinguisher located nearby?",
        "(12) Are working smoke detectors installed at the work site?",
        "(13) Is the work area private and free of intrusion?",
        "(14) Are files and data secure?",
        "(15) Are first aid supplies readily accessible and adequate?",
        "(16) Are office furniture and equipment ergonomically correct?",
        "(17) Desk: 29” high?",
        "(18) Chairs: Sturdy and adjustable (90° at knees, feet flat on floor,
15° back tilt)\n     with backrest and casters appropriate for floor surface?
",
        "(19) Keyboard: In line with wrist and forearm position?",
        "(20) Monitor: 20-14” from eyes: Top of screen slightly below eye
level?",
        "(21) Are work materials and equipment in a secure place that can be
protected from damage or misuse?",
        "(22) Are there security requirements in place to protect confidentiality
and security of\n     company information and computer systems?
"
    };

    string employeeName, date, managerName, remoteWorkAddress,
workAreaDescription;
    int countY = 0, countN = 0; // Initialize counters

    // Get the current entry number
    int entryNumber = getCurrentEntryNumber();
    entryNumber++; // Increase the entry number

    // Display the entry number
    cout << "\n-----------------------------------------------" << endl;
    cout << "Entry Number: " << entryNumber << endl;
    cout << "-----------------------------------------------\n" << endl;
    cin.ignore(); // Clear the newline character left in the input buffer

    // Collect user information
    do {
        cout << "Employee                  : ";
        getline(cin, employeeName); // Read the employee's name
        if (employeeName == "0") return; // Exit if '0' is entered
    } while (employeeName.empty());

    do {
        cout << "Date                      : ";
        getline(cin, date); // Read the date
```

```
        if (date == "0") return; // Exit if '0' is entered
    } while (date.empty());

    do {
        cout << "Manager                    : ";
        getline(cin, managerName); // Read the manager's name
        if (managerName == "0") return; // Exit if '0' is entered
    } while (managerName.empty());

    do {
        cout << "Remote work site address: ";
        getline(cin, remoteWorkAddress); // Read the remote work site address
        if (remoteWorkAddress == "0") return; // Exit if '0' is entered
    } while (remoteWorkAddress.empty());

    do {
        cout << "Description of work area: ";
        getline(cin, workAreaDescription); // Read the description of the work
area
        if (workAreaDescription == "0") return; // Exit if '0' is entered
    } while (workAreaDescription.empty());

    cout << endl; // Print a blank line
    cout << "-------------------------------------------------------------------
-----------------------------------------------\n" << endl;

    // Open a file to write the checklist data
    ofstream outFile("SafetyChecklist.txt", ios::app);

    // Write the entry number to the file
    outFile << "\n-----------------------------------------------" << endl;
    outFile << "Checklist Number: " << entryNumber << endl;
    outFile << "-----------------------------------------------\n" << endl;


    // Write user information to file
    outFile << "Home Office Safety Checklist\n\n";
    outFile << "Employee                   : " << employeeName << endl;
    outFile << "Date                       : " << date << endl;
    outFile << "Manager                    : " << managerName << endl;
    outFile << "Remote work site address   : " << remoteWorkAddress << endl;
    outFile << "Description of work area   : " << workAreaDescription << endl;
    outFile << endl; // Print a blank line in the txt file

    // Write the table header
    outFile << "-------------------------------------------------------------------
-----------------------------------------------" << endl;
    outFile << left << setw(110) << "Question" << "Answer" << endl;
    outFile << "-------------------------------------------------------------------
-----------------------------------------------\n" << endl;

    // Iterate (go) through the array of questions and ask each one
    for (int i = 0; i < 22; i++) {
        outFile << left << setw(110) << questions[i];
        askQuestion(&questions[i], &outFile, &countY, &countN); // Ask the
question and write the answer to the file
```

```
        outFile << endl; // Print a blank line in the txt file
    }

    // Write summary information
    outFile << "-------------------------------------------------------------------
-------------------------------------------------" << endl;
    outFile << "    Total number of Yes: " << countY << endl;
    outFile << "    Total number of No : " << countN << endl;
    outFile << "-------------------------------------------------------------------
-------------------------------------------------" << endl;

    // Add the summary to the file based on the number of 'Yes' and 'No' answers
    if (countN == 0) {
        outFile << "    Summary: The workspace is fully suitable for remote
work." << endl;
    }
    else if (countN <= 3) {
        outFile << "    Summary: The workspace is mostly suitable for remote work
but has some minor issues to address." << endl;
    }
    else if (countN <= 6) {
        outFile << "    Summary: The workspace has several issues that should be
addressed before it can be considered suitable." << endl;
    }
    else {
        outFile << "    Summary: The workspace is currently not suitable for
remote work due to significant safety concerns." << endl;
    }

    outFile << "-------------------------------------------------------------------
-------------------------------------------------" << endl;

    // Close the file after writing all data
    outFile << endl; // Print a blank line in the txt file
    outFile <<
"xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" << endl;

    outFile.close();

    // Update the entry number file
    updateEntryNumber(entryNumber);

    cout << "-------------------------------------------------------------------
-------------------------------------------------" << endl;
    cout << "\nEntry completed and saved to SafetyChecklist.txt" << endl;
    //Wait for user to continue
    cout << "\nEnter any key to continue...";
    cin.ignore();
    cin.get();
}


/*
```

```
 // Function to ask a question, get the user's answer, and write the question and
answer to a file using pointers

  Asks a question to the user, retrieves their answer, and writes the question
and answer to a file.

  The function performs the following tasks:
  - Prompts the user with a question and requests a Yes/No answer.
  - Validates the user's answer to ensure it is either 'Y', 'Yes', 'N', or 'No'
(case insensitive).
  - Converts the answer to a consistent format ('Yes' or 'No').
  - Updates counters for 'Yes' and 'No' answers.
  - Writes the question and user's answer to the specified output file.

  :param question - A pointer to a string containing the question to be asked.
  :param outFile -  A pointer to an ofstream object used to write the question
and answer to a file.
  :param countY -  A pointer to an integer that keeps track of the number of
'Yes' answers.
  :param countN - A pointer to an integer that keeps track of the number of 'No'
answers.
 */
void askQuestion(string* question, ofstream* outFile, int* countY, int* countN) {
    string answer; // Variable to store the user's answer
    cout << *question << "\n\n     (Y-Yes/n-No): "; // Prompt the user with the
question and ask for a Yes/No answer
    cin >> answer;

    // Validate the answer
    while (!isValidAnswer(answer)) {
        cout << "\n     Invalid answer. Please enter (Y-Yes or N-No) [case
insensitive]: ";
        cin >> answer;
    }

    // This is done to ensure consistency throughout the answers in the checklist
    if (answer == "YES" || answer == "Yes" || answer == "Y" || answer == "y") {
        answer = "Yes";
        (*countY)++;
    }
    else {
        answer = "No";
        (*countN)++;
    }

    *outFile << left << setw(3) << answer << endl; // Write the question and the
user's answer  to the file, followed by a newline
    cout << endl; // Print a blank line
}

/**
 // Function to generate and display a report from SafetyChecklist.txt
  Generates and displays a report from the SafetyChecklist.txt file.

  This function performs the following tasks:
  - Opens the SafetyChecklist.txt file for reading.
```

```
  - Reads and displays each line of the file on the console.
  - Provides error messages if the file cannot be opened.
  - Waits for user input before returning to the previous menu.

  :pre - The SafetyChecklist.txt file should exist and contain valid data.
  :post - The console displays the contents of SafetyChecklist.txt or an error
message if the file cannot be opened.
 */
void generateSafetyReport() {
    ifstream in_File("SafetyChecklist.txt");
    string line;

    if (in_File.is_open()) {
        cout << "\nSafety Checklist Report:\n" << endl;
        while (getline(in_File, line)) {
            cout << line << endl;
        }
        in_File.close(); // Close the file after reading
    }
    else {
        cout << "Unable to open the SafetyChecklist.txt file.\n" << endl;
        cout << "Possible Reason 1: No entry for Safety Checklist (option[1]).
Please Complete it atleast once.\n" << endl;
        cout << "Possible Reason 2: SafetyChecklist.txt file has not been
created.\n" << endl;

    }
    //Wait for user to continue
    cout << "\nEnter any key to return...";
    cin.ignore();
    cin.get();
    system("cls");
}

/*
 // Function to get the current entry number by reading from a file
  Gets the current entry number by reading from the EntryNumber.txt file.

  This function performs the following tasks:
  - Opens the EntryNumber.txt file for reading.
  - Reads the entry number from the file.
  - Closes the file after reading.

  :return int - The current entry number read from the file. If the file is empty
or cannot be opened, returns 0.
 */

int getCurrentEntryNumber() {
    ifstream inFile("EntryNumber.txt");
    int entryNumber = 0; // Initialize entry number to 0

    if (inFile.is_open()) {
        inFile >> entryNumber;  // Read the entry number from the file
        inFile.close(); // Close the file after reading
    }
    return entryNumber; // Return the entry number
```

```
}

/*
 // Function to update the entry number in the file
  Updates the entry number in the EntryNumber.txt file.

  This function performs the following tasks:
  - Opens the EntryNumber.txt file for writing.
  - Writes the new entry number to the file.
  - Closes the file after writing.
  - Displays an error message if the file cannot be opened.

  :param newEntryNumber - The new entry number to be written to the file.
 */
void updateEntryNumber(int newEntryNumber) {
    ofstream outFile("EntryNumber.txt");
    if (outFile.is_open()) {
        outFile << newEntryNumber; // Write the updated entry number to the file
        outFile.close(); // Close the file after writing
    }
    else {
        cout << "Unable to update entry number file." << endl; // Error message
if file cannot be opened
        //Wait for user to continue
        cout << "\nEnter any key to continue...";
        cin.ignore();
        cin.get();
    }
}



/**
 // Function to check if the user's answer is valid
  Checks if the user's answer is valid.

  This function validates the user's response to ensure it matches expected
values for Yes or No.

  :param answer - The user's answer to validate.
  :return bool - True if the answer is one of the valid responses (Yes/No), false
otherwise.
 */
bool isValidAnswer(string& answer) {
    return answer == "Yes" || answer == "YES" || answer == "yes" || answer == "y"
|| answer == "Y" ||
        answer == "No" || answer == "NO" || answer == "no" || answer == "n" ||
answer == "N";
}

bool case1CompletedFirst = false; // Used to check if Case 1 was done first (only
for the first time)

/*
 // Function to handle Productivity Tracking System
  Handles the Productivity Tracking System menu and its options.
```

```
  This function displays the Productivity Tracking System menu with options to:
  1. Log work activity and track productivity metrics.
  2. Generate a report based on recorded work activities.
  0. Return to the main menu.

  It ensures that users complete work logging before generating a report.

  :pre - The global variable `case1CompletedFirst` is used to track whether work
activity logging has been completed.
  :post - Depending on user selection, the function calls `workLog()` to log
activities, `generateReport()` to generate reports,
          or returns to the main menu. The console displays appropriate messages
based on the user's actions and status.
 */

void productivityTrackingSystem() {

    int subChoice; // Variable to store the user's submenu choice

    do {
        // Display the Productivity Tracking System menu
        cout << "Productivity Tracking System Menu selected. [2]" << endl;
        cout << "\nProductivity Tracking System Menu" << endl;
        cout << "----------------------------------------------------------------
---\n" << endl;
        cout << "Log Work Activity & Track Productivity Metrics ----------------
[1]\n" << endl;
        cout << "Generate Report -----------------------------------------------
[2]\n" << endl;
        cout << "\nReturn to Main Menu -------------------------------------------
--[0]\n" << endl;
        cout << "----------------------------------------------------------------
---" << endl;
        cout << "Please select an option (1, 2, or 0): ";

        // Validate user input for submenu choice
        while (!(cin >> subChoice) || (subChoice < 0 || subChoice > 3)) {
            cin.clear(); // Clear error flags
            cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Discard
invalid input
            cout << "\nInvalid input. Please enter a number (1, 2, or 0): ";
        }

        // Handle user selection
        switch (subChoice) {
        case 1:
            system("cls"); // Clear screen
            workProductivityLog(); // Call the function to log work activities
and track productivity metrics
            system("cls"); // Clear screen after work log

            case1CompletedFirst = true; // Mark that work logging has been
completed

            break;
```

```
        case 2:
            if (case1CompletedFirst == true) { // Check if work activity has been
logged

                system("cls"); // Clear screen
                cout << "Report List" << endl;
                generateProductivityReport(); // Call the function to generate
the productivity report
                system("cls"); // Clear screen after generating the report
            }

            else {
                system("cls"); // Clear screen
                cout << "Posibble Reason 1: No Entry Recorded \n"
                    << "\nPlease Login your Work Activity and Complete Track
Productivity Metrics (Option [1])\n"
                    << "\nBefore Generating Report\n" << endl;
                //Wait for user to continue
                cout << "\nEnter any key to return...";
                cin.ignore();
                cin.get();
                system("cls"); // Clear screen after message
            }
            break;
        case 0:
            cout << "Returning to Main Menu..." << endl; // Inform the user that
they are  returned to the Main menu.
            break;
        }

    } while (subChoice != 0); // Repeat until the user selects the option to exit
}



/*
 // Function to handle WorkLog
  Collects and logs work activities and productivity metrics.

  This function:
  - Creates or opens the WorkLog.txt file in append mode.
  - Retrieves and displays the current entry number.
  - Collects basic information such as name, contact, date, department, and
supervisor.
  - Collects details of each activity, including start and end times, status, and
notes.
  - Calculates total scheduled time and logs productivity metrics for each
activity.
  - Writes all collected data to the WorkLog.txt file with proper formatting.
  - Updates the work entry number.

  :pre - The `EntryNumber.txt` file must exist and contain a valid integer to
track the current entry number.
  :post - The WorkLog.txt file is updated with new entries, including work
activities and productivity metrics.
```

```
 */
void workProductivityLog() {
    // Create WorkLog.txt file in append mode
    ofstream workLogFile("WorkLog.txt", ios::app);

    string name, contact, date, department, supervisor, experience;
    int numActivities;

    // Retrieve and increase entry number
    int workNumber = getCurrentWorkNumber() + 1;


    // Display current entry number
    cout << "----------------------\n";
    cout << "WORK LOG - ENTRY " << workNumber << "\n";
    cout << "----------------------\n\n";
    cin.ignore();// Ignore leftover newline characters

    // Collecting basic information
    do {

        cout << "Name         : ";
        getline(cin, name); // Read the name
        if (name == "0") return; //if user enter 0, return
    } while (name.empty());

    do {
        cout << "Contact No   : ";
        getline(cin, contact); // Read the contact

    } while (contact.empty());

    do {
        cout << "Today's Date : ";
        getline(cin, date); // Read the date

    } while (date.empty());

    do {
        cout << "Department   : ";
        getline(cin, department); // Read the department

    } while (department.empty());

    do {
        cout << "Supervisor   : ";
        getline(cin, supervisor); // Read the supervisor

    } while (supervisor.empty());

    // Collect the number of activities with input validation
    while (true) {
        cout << "\nEnter the number of activities for today: ";
        cin >> numActivities;

        // Check if the input is a valid integer
```

```
        if (cin.fail() || numActivities <= 0) {
            cout << "ERROR: Please enter a valid positive integer for the number
of activities: " << endl;
            cin.clear(); // Clear the error flag on cin
            cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Discard
invalid input
        }
        else {
            cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Discard
remaining input
            break; // Exit loop if input is valid
        }
    }

    // Check if file was opened successfully
    if (!workLogFile.is_open()) {
        cerr << "Error: Unable to open WorkLog.txt." << endl;
        return;
    }

    // Writing the entry number and basic information with better formatting
    workLogFile << "\nWORK LOG - ENTRY " << workNumber << "\n";
    workLogFile << "-----------------------------------------------------------
-----------------------------------------\n";
    workLogFile << left << setw(24) << "NAME" << setw(25) << "CONTACT" <<
setw(20) << "TODAY'S DATE" << setw(20) << "DEPARTMENT" << setw(11) <<
"SUPERVISOR\n";
    workLogFile << "-----------------------------------------------------------
-----------------------------------------\n";
    workLogFile << left << setw(24) << name << setw(25) << contact << setw(20) <<
date << setw(20) << department << setw(11) << supervisor << "\n\n";

    workLogFile << "-----------------------------------------------------------
-----------------------------------------\n";
    workLogFile << left << setw(45) << "ACTIVITY" << setw(12) << "START TIME" <<
setw(12) << "END TIME" << setw(10) << "TOTAL" << setw(15) << "STATUS" <<
"NOTES\n";
    workLogFile << "-----------------------------------------------------------
-----------------------------------------\n";

    // Initialize totalschtime accumulator
    int totalschTimeMinutes = 0;

    // Allocate memory for the activities array dynamically
    /*
        Dynamic memory allocation is used here because the number of activities
        is not known at compile time. Allocating memory dynamically allows us
        to create an array of the exact size needed based on user input.
    */
    string* activities = new string[numActivities];

    // Loop through activities and Collect details for each activity
    for (int i = 0; i < numActivities; i++) {
        string activity, startTime, endTime, status, notes;
        int totalMinutes;
```

```
        cout << "\n-----------------" << endl;
        cout << "Enter Activity " << (i + 1) << ": ";
        getline(cin, activity);
        activities[i] = activity; // Store activity in array
        cout << "-----------------------------------------------------------------
--\n" << endl;

        // Collect and validate start time
        while (true) {
            cout << "Enter Start Time (24-hour format) (HH:MM): ";
            getline(cin, startTime);
            if (isValidTime(startTime)) {
                break;
            }
            cout << "Invalid time format. Please enter time in HH:MM format.\n"
<< endl;
        }

        // Collect and validate end time
        while (true) {
            cout << "Enter End Time (24-hour format) (HH:MM): ";
            getline(cin, endTime);
            if (isValidTime(endTime)) {
                break;
            }
            cout << "Invalid time format. Please enter time in HH:MM format.\n"
<< endl;
        }

        cout << "\nEnter Status (e.g., In Progress, Complete, Not Started): ";
        getline(cin, status);
        cout << "Enter Notes: ";
        getline(cin, notes);
        cout << "-----------------------------------------------------------------
--\n" << endl;

        // Calculate the total time in minutes
        int startMinutes = timeToMinutes(startTime);
        int endMinutes = timeToMinutes(endTime);
        totalMinutes = endMinutes - startMinutes;

        if (totalMinutes < 0) {
            // Adjust for cases where the end time is after midnight
            totalMinutes += 24 * 60;
        }

        // Accumulate the total scheduled time in minutes
        totalschTimeMinutes += totalMinutes;

        // Convert total time and accumulated time back to HH:MM format
        string totalTimeStr = minutesToTime(totalMinutes);
        string totalschTimeStr = minutesToTime(totalschTimeMinutes);

        // Write the activity to the file with better formatting
        workLogFile << left << setw(45) << activity << setw(12) << startTime <<
setw(12) << endTime
```

```
            << setw(10) << totalTimeStr << setw(15) << status << notes << "\n";
    }

    // Write total scheduled time to file
    workLogFile << "\n-------------------------------------------------------------
-----------------------------------------\n";
    workLogFile << "Total Scheduled Time : " <<
minutesToTime(totalschTimeMinutes) << "\n";
    workLogFile << "-------------------------------------------------------------
-----------------------------------------\n\n";

    system("cls");


    // The Start OF Productivity Tracking
    workLogFile << "\nProductivity Tracking\n";
    workLogFile << "-------------------------------------------------------------
-----------------------------------------------------------------------
------\n";
    workLogFile << left << setw(30) << "ACTIVITY" << setw(12) << "CATEGORY" <<
setw(14) << "FOCUS LEVEL" << setw(15) << "DISTRACTIONS"
        << setw(15) << "PRODUCTIVE?" << setw(30) << "EVALUATION" << setw(22) <<
"SATISFACTION" << setw(9) << "FEEDBACK\n";
    workLogFile << "-------------------------------------------------------------
-----------------------------------------------------------------------
------\n";

    int productiveTasks = 0;
    int nonProductiveTasks = 0;

    cout << "---------------------" << endl;
    cout << "Productivity Tracking" << endl;
    cout << "---------------------" << endl;
    cout << "\n-------------------------------------------------------------------
---------------------------------------\n";

    // Collect productivity details for each activity
    for (int i = 0; i < numActivities; i++) {
        string category, productive, priority, evaluate, satisfaction, feedback;
        int focusLevel, distractions;

        do {
            cout << "\nEnter Category for activity \"" << activities[i] << "\"
(e.g., Emailing, Coding, Meetings): ";
            getline(cin, category);
        } while (category.empty());

        cout << "\nRate the focus level required for \"" << activities[i] << "\"
on a scale of 1-5: ";
        while (!(cin >> focusLevel) || focusLevel < 1 || focusLevel > 5) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "\nInvalid Input. Please enter a number between 1 and 5: ";
        }
```

```
        cout << "\nHow often were you distracted during \"" << activities[i] <<
"\" on a scale of 1-5: ";
        while (!(cin >> distractions) || distractions < 1 || distractions > 5) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "\nInvalid Input. Please enter a number between 1 and 5: ";
        }

        cout << "\nWas the activity \"" << activities[i] << "\" productive?
(Yes/No): ";
        cin.ignore();
        getline(cin, productive);

        // Validate answer
        while (!isValidAnswer(productive)) {
            cout << "Invalid input. Please answer (Y-Yes or N-No): ";
            getline(cin, productive);
        }

        // This is done to ensure consistency throughout the answers in the
checklist and Count productive/non-productive tasks
        if (productive == "YES" || productive == "Yes" || productive == "Y" ||
productive == "y") {
            productive = "Yes";
            productiveTasks++;
        }
        else {
            productive = "No";
            nonProductiveTasks++;
        }

        // Collect additional feedback
        do {
            cout << "\nHow would you evaluate the activity \"" << activities[i]
<< "\"? (e.g., highly productive, could have been better): ";
            getline(cin, evaluate);
        } while (evaluate.empty());

        do {
            cout << "\nHow satisfied were you with the completion of \"" <<
activities[i] << "\"? (e.g., very satisfied, somewhat satisfied, not satisfied):
";
            getline(cin, satisfaction);
        } while (satisfaction.empty());

        do {
            // Additional open-ended feedback question
            cout << "\nPlease provide any additional thoughts on the activity \""
<< activities[i] << "\": ";
            getline(cin, feedback);
        } while (feedback.empty());
        cout << "\n-----------------------------------------------------------------
----------------------------------------\n";

        // Write productivity details to file with formatting
```

```
        workLogFile << left << setw(30) << activities[i] << setw(12) << category
<< setw(14) << focusLevel
            << setw(15) << distractions << setw(15) << productive << setw(30) <<
evaluate << setw(22) << satisfaction << setw(20) << feedback << "\n";
    }

    //Ask for user's overall experience
    cout << "\nHow was your overall task experience? : ";
    cin >> experience;

    // Write productivity summary to file
    workLogFile << "\n-----------------------------------------------------------
-------------------------------------------------------------------------------
--------\n";
    workLogFile << "Productive Tasks     : " << productiveTasks << "\n";
    workLogFile << "Non-Productive Tasks : " << nonProductiveTasks << "\n";
    workLogFile << "Total Number of Task Completed: " << numActivities << "\n";
    workLogFile << "-----------------------------------------------------------
-------------------------------------------------------------------------------
------\n";
    workLogFile << "Overall Experience : " << experience << endl;
    workLogFile << "-----------------------------------------------------------
-------------------------------------------------------------------------------
------\n\n\n";
    workLogFile <<
"xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx\n" << endl;

    // Free the allocated memory
    delete[] activities;

    // Close the file and update work entry number
    workLogFile.close();
    updateWorkNumber(workNumber);

    cout << "\nWork log saved successfully to WorkLog.txt!" << endl;
    //Wait for user to continue
    cout << "\nEnter any key to continue...";
    cin.ignore();
    cin.get();
    system("cls");
}


/*
 // Function to check if the time format is valid
  Checks if the given time string is in a valid HH:MM format.

  :param timeStr - The time string to check.
  :return - True if the time string is valid, otherwise false.
 */
bool isValidTime(const string& timeStr) {
    // Check if the length of the string is 5 and if the third character is a
colon
    if (timeStr.length() != 5 || timeStr[2] != ':')
```

```
    {
        return false; // Invalid format
    }

    // Extract hours and minutes from the string
    int hours = stoi(timeStr.substr(0, 2)); // Get hours from the first two
characters
    int minutes = stoi(timeStr.substr(3, 2)); // Get minutes from the last two
characters

    // Check if hours are within the range [0, 23] and minutes are within the
range [0, 59]
    return hours >= 0 && hours < 24 && minutes >= 0 && minutes < 60;
}

/*
 // Function to convert HH:MM to total minutes
  Converts a time string in HH:MM format to the total number of minutes.

  :param timeStr - The time string to convert.
  :return - The total number of minutes.
 */
 // Function to convert HH:MM to total minutes
int timeToMinutes(const string& timeStr) {
    int hours, minutes;
    char colon;
    istringstream timeStream(timeStr);
    timeStream >> hours >> colon >> minutes;
    return hours * 60 + minutes;
}

/*
 // Function to convert total minutes to HH:MM format
  Converts total minutes into a time string in HH:MM format.

  :param totalMinutes - The total number of minutes to convert.
  :return - The time string in HH:MM format.
 */
string minutesToTime(int totalMinutes) {
    int hours = totalMinutes / 60;
    int minutes = totalMinutes % 60;
    ostringstream timeStream;
    timeStream << setw(2) << setfill('0') << hours << ":"
        << setw(2) << setfill('0') << minutes;
    return timeStream.str();
}

/*
 // Function to get the current work number from the file
  Retrieves the current work number from the WorkNumber.txt file.

  :return int - The current work number. If the file cannot be opened, returns 0.
 */
int getCurrentWorkNumber() {
    int workNumber = 0;
    ifstream workFile("WorkNumber.txt");
```

```
    if (workFile.is_open()) {
        workFile >> workNumber; // Read the work number from the file
        workFile.close();
    }
    else {
        cerr << "Error: Unable to open WorkNumber.txt. Starting from entry 1." <<
endl;
    }
    return workNumber; // Return the current work number
}

/*
 // Function to update the login number file with the new login number
  Updates the WorkNumber.txt file with the new work number.

  :param newWorkNumber - The new work number to write to the file.
 */
void updateWorkNumber(int newWorkNumber) {
    ofstream workFile("WorkNumber.txt");

    if (workFile.is_open()) {
        workFile << newWorkNumber; // Write the new work number to the file
        workFile.close();
    }
    else {
        cerr << "Error: Unable to update WorkNumber.txt." << endl;
        //Wait for user to continue
        cout << "\nEnter any key to continue...";
        cin.ignore();
        cin.get();
    }

}

/*
 // Function to generate and display a report of logged activities
 Generates and displays a report of logged activities from WorkLog.txt.
 */
void generateProductivityReport() {
    ifstream in_File("WorkLog.txt");
    string line;

    if (in_File.is_open()) {
        cout << "\nGenerated Report:\n" << endl;
        while (getline(in_File, line)) {
            cout << line << endl;
        }
        in_File.close(); // Close the file after reading

    }
    else {
        cout << "Unable to open the Work Log file." << endl;
    }

    //Wait for user to continue
```

```
    cout << "\nEnter any key to continue...";
    cin.ignore();
    cin.get();
    system("cls");
}
```

3 SAMPLE OUTPUT

3.1 Main Menu

The main menu is displayed once the program is loaded.

The user can enter:

1) Access the Remote Work Readiness Menu
2) Productivity Tracking System Menu
0) Exit (Exit the Program)

```
Welcome to the Remote Work Readiness and Productivity Tracking System
------------------------------------------------------------------

Remote Work Readiness Menu ----------- [1]

Productivity Tracking System Menu ---- [2]


Exit ------------------------------- [0]

------------------------------------------------------------------
Please select an option (1, 2, or 0):
```

3.1.1 Data Validation

When the user enters a non-number and not in the range, the program will Display "**Invalid input. Please enter a number (1, 2, or 0):** ", until the user enters either (1, 2, or 0).

```
Welcome to the Remote Work Readiness and Productivity Tracking System
------------------------------------------------------------------

Remote Work Readiness Menu ----------- [1]

Productivity Tracking System Menu ---- [2]


Exit ------------------------------- [0]

------------------------------------------------------------------
Please select an option (1, 2, or 0): 8

Invalid input. Please enter a number (1, 2, or 0): k

Invalid input. Please enter a number (1, 2, or 0): -

Invalid input. Please enter a number (1, 2, or 0):
```

3.2 Remote Work Readiness Menu

The Remote Work Readiness Menu is displayed when the user enters [1]

The user can enter:

1) Remote Work Readiness Safety Checklist
2) Generate Safety Report

0) Return to the Main Menu

```
Remote Work Readiness Menu selected. [1]

Remote Work Readiness Menu
------------------------------------------

Remote Work Readiness safety Checklist --[1]

Generate Report -------------------------[2]


Return to Main Menu --------------------[0]

------------------------------------------
Please select an option (1, 2, or 0):
```

3.2.1 Data Validation

When the user enters a non-number and not in the range, the program will Display "**Invalid input. Please enter a number (1, 2, or 0):** ", until the user enters either (1, 2, or 0).

```
Remote Work Readiness Menu selected. [1]

Remote Work Readiness Menu
------------------------------------------

Remote Work Readiness safety Checklist --[1]

Generate Report -------------------------[2]


Return to Main Menu --------------------[0]

------------------------------------------
Please select an option (1, 2, or 0): -

Invalid input. Please enter a number (1, 2, or 0): 6

Invalid input. Please enter a number (1, 2, or 0): k

Invalid input. Please enter a number (1, 2, or 0): -4

Invalid input. Please enter a number (1, 2, or 0):
```

3.3 Remote Work Readiness Function

The Remote Work Readiness function is displayed when the user enters [1].

```
Remote Work Readiness selected.

-----------------------------------------------
Entry Number: 1
-----------------------------------------------

Employee                  :
```

3.3.1 Sample of Remote Work Readiness Function

```
Remote Work Readiness selected.

-----------------------------------------
Entry Number: 1
-----------------------------------------

Employee                : Thomas Gan
Date                    : 27 September 2024
Manager                 : Mr. Tom
Remote work site address: No. 12, Jalan Meranti 2, Taman Seri Damai, 40100 Shah Alam, Selangor
Description of work area: Backend Development

-------------------------------------------------------------------------------------------------

(1) Is the work space free from excessive noise?

     (Y-Yes/n-No): y

(2) Is adequate lighting (side or rear) provided at the work station?

     (Y-Yes/n-No): no

(3) Is all electrical equipment free of recognized hazards that could cause physical harm
     (frayed wires running through walls, exposed wires fixed to the ceiling)?

     (Y-Yes/n-No): Yes

(4) Is electrical system adequate for office equipment?

     (Y-Yes/n-No):
```

3.3.2 Data Validation for Personal Information

If the user leaves the section blank, the program will ask the user to re-enter again.

```
Remote Work Readiness selected.

------------------------------------------------
Entry Number: 1
------------------------------------------------

Employee                    :
Employee                    :
Employee                    :
```

3.3.3 Data Validation for Safety Checklist

If the user enters an invalid answer, the program will display “Invalid answer. Please enter (Y-Yes or N-No) [case insensitive]:” and prompt the user to re-enter a valid answer.

```
(4) Is electrical system adequate for office equipment?

     (Y-Yes/n-No): 56

     Invalid answer. Please enter (Y-Yes or N-No) [case insensitive]: trfrfasf

     Invalid answer. Please enter (Y-Yes or N-No) [case insensitive]: --++?

     Invalid answer. Please enter (Y-Yes or N-No) [case insensitive]:
```

3.3.4 Notification for Completion of Safety Checklist

Once the user has successfully answered the questions, the program will notify the user that the entry is completed and recorded to the **SafetyChecklist.txt**.

Furthermore, the program will wait for the user input to continue the program.

```
(22) Are there security requirements in place to protect confidentiality and security of
     company information and computer systems?

     (Y-Yes/n-No): n

------------------------------------------------------------------------------------------

Entry completed and saved to SafetyChecklist.txt

Enter any key to continue...
```

3.4 Generate Safety Report Function

If the user enters option [2] to generate a Safety Report, the data recorded from **SafetyChecklist.txt** will be read and displayed.

```
Generating Report from SafetyChecklist.txt...


Safety Checklist Report:


-----------------------------------------------
Checklist Number: 1
-----------------------------------------------

Home Office Safety Checklist

Employee                  : Thomas Gan
Date                      : 27 September 2024
Manager                   : Mr. Tom
Remote work site address : No. 12, Jalan Meranti 2, Taman Seri Damai, 40100 Shah Alam, Selangor
Description of work area : Backend Development

------------------------------------------------------------------------------------------------------------------------
Question                                                                                                   Answer
------------------------------------------------------------------------------------------------------------------------

(1) Is the work space free from excessive noise?                                                           Yes

(2) Is adequate lighting (side or rear) provided at the work station?                                      Yes

(3) Is all electrical equipment free of recognized hazards that could cause physical harm
     (frayed wires running through walls, exposed wires fixed to the ceiling)?                             Yes

(4) Is electrical system adequate for office equipment?                                                    Yes

(5) Is electrical equipment grounded?                                                                      Yes

(6) Are surge protectors properly installed?                                                               No

(7) Are aisles, doorways and floors free of obstructions to permit visibility and movement?                No

(8) Is there an exit that allows prompt exiting?                                                           No

(9) Are phone lines, electrical cords and extension wires secured under a desk or along a baseboard?        No
```

```
(10) Is the office space neat and clean?                                                                   No

(11) Is a working fire extinguisher located nearby?                                                        No

(12) Are working smoke detectors installed at the work site?                                               Yes

(13) Is the work area private and free of intrusion?                                                       Yes

(14) Are files and data secure?                                                                            Yes

(15) Are first aid supplies readily accessible and adequate?                                               No

(16) Are office furniture and equipment ergonomically correct?                                             No

(17) Desk: 29ö high?                                                                                       No

(18) Chairs: Sturdy and adjustable (90░ at knees, feet flat on floor, 15░ back tilt)
     with backrest and casters appropriate for floor surface?                                              No

(19) Keyboard: In line with wrist and forearm position?                                                    No

(20) Monitor: 20-14ö from eyes: Top of screen slightly below eye level?                                    Yes

(21) Are work materials and equipment in a secure place that can be protected from damage or misuse?       Yes

(22) Are there security requirements in place to protect confidentiality and security of
     company information and computer systems?                                                             Yes

------------------------------------------------------------------------------------------------------------------------
    Total number of Yes: 11
    Total number of No : 11
------------------------------------------------------------------------------------------------------------------------
    Summary: The workspace is currently not suitable for remote work due to significant safety concerns.
------------------------------------------------------------------------------------------------------------------------

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

Enter any key to return...
```

3.4.1 Validation of Generate Safety Report Function

The program will display an error message if the user selects option [2] before completing option [1] and will kindly ask the user to complete the Safety Checklist before generating the report.

```
Please complete Safety Checklist (option[1]) first, before Generating Report

Enter any key to return...
```

3.5 Productivity Tracking System Function

The Productivity Tracking System Menu is displayed when the user enters [2]

The user can enter:

1) Log Work Activity & Track Productivity Metrics
2) Generate Productivity Report
0) Return to the Main Menu

```
Productivity Tracking System Menu selected. [2]

Productivity Tracking System Menu
-----------------------------------------------------------------

Log Work Activity & Track Productivity Metrics ----------------[1]

Generate Report -----------------------------------------------[2]


Return to Main Menu -------------------------------------------[0]

-----------------------------------------------------------------
Please select an option (1, 2, or 0):
```

3.5.1 Data Validation

When the user enters a non-number and is not in the range, the program will Display "**Invalid input. Please enter a number (1, 2, or 0):** ", until the user enters either (1, 2, or 0).

```
Productivity Tracking System Menu selected. [2]

Productivity Tracking System Menu
------------------------------------------------------------

Log Work Activity & Track Productivity Metrics ----------------[1]

Generate Report -----------------------------------------------[2]


Return to Main Menu -------------------------------------------[0]

------------------------------------------------------------
Please select an option (1, 2, or 0): -1

Invalid input. Please enter a number (1, 2, or 0): 7

Invalid input. Please enter a number (1, 2, or 0): l

Invalid input. Please enter a number (1, 2, or 0): -

Invalid input. Please enter a number (1, 2, or 0):
```

3.6 Log Work Activity & Track Productivity Metrics

The Log Work Activity & Track Productivity Metrics function is displayed when the user enters [1].

An error message will pop out if the file, **WorkNumber.txt** does not exist yet to save the number of entries.

```
Error: Unable to open WorkNumber.txt. Starting from entry 1.
--------------------
WORK LOG - ENTRY 1
--------------------

Name         :
```

3.6.1 Sample of Log Work Activity

```
--------------------
WORK LOG - ENTRY 1
--------------------

Name         : Aisyah Rahman
Contact No   : +60 12-345 6789
Today's Date : 01/09/2024
Department   : Human Resources
Supervisor   : Mr. Ahmad Sulaiman

Enter the number of activities for today: 3

-----------------
Enter Activity 1: Client Meeting
-----------------------------------------------------------------

Enter Start Time (24-hour format) (HH:MM): 00:00
Enter End Time (24-hour format) (HH:MM): 02:00

Enter Status (e.g., In Progress, Complete, Not Started): Completed
Enter Notes: The meeting went well
-----------------------------------------------------------------


-----------------
Enter Activity 2: Report Writing
-----------------------------------------------------------------

Enter Start Time (24-hour format) (HH:MM): 02:00
Enter End Time (24-hour format) (HH:MM): 05:00

Enter Status (e.g., In Progress, Complete, Not Started): In progress
Enter Notes: Struggling
-----------------------------------------------------------------


-----------------
Enter Activity 3: Team Collaboration
-----------------------------------------------------------------

Enter Start Time (24-hour format) (HH:MM): 08:00
Enter End Time (24-hour format) (HH:MM): 10:00

Enter Status (e.g., In Progress, Complete, Not Started): Complete
Enter Notes: Productive
```

3.6.1 Data Validation for Personal Information

If the user leaves the section blank, the program will ask the user to re-enter again.

```
--------------------
WORK LOG - ENTRY 3
--------------------

Name          :
Name          :
Name          :
```

3.6.2 Data Validation for number of activities

If the user enters an invalid answer, the program will display "**ERROR: Please enter a valid positive integer for the number of activities**:" until the user enters a valid answer.

```
Enter the number of activities for today: k
ERROR: Please enter a valid positive integer for the number of activities:

Enter the number of activities for today: r
ERROR: Please enter a valid positive integer for the number of activities:

Enter the number of activities for today: 0
ERROR: Please enter a valid positive integer for the number of activities:

Enter the number of activities for today: -1
ERROR: Please enter a valid positive integer for the number of activities:

Enter the number of activities for today:
```

3.6.3 Data Validation for Time

If the user enters an incorrect time format, the system will display "**Invalid time format. Please enter time in (HH: MM) format**." Until the user enters a valid time format.

```
-----------------
Enter Activity 1: Help me
------------------------------------------------------------

Enter Start Time (24-hour format) (HH:MM): llss
Invalid time format. Please enter time in HH:MM format.

Enter Start Time (24-hour format) (HH:MM): 06:90
Invalid time format. Please enter time in HH:MM format.

Enter Start Time (24-hour format) (HH:MM): 7:89
Invalid time format. Please enter time in HH:MM format.

Enter Start Time (24-hour format) (HH:MM):
```

3.6.4 Sample of Log Work Activity

```
--------------------
Productivity Tracking
--------------------

---------------------------------------------------------------------------------------------------------

Enter Category for activity "Client Meeting" (e.g., Emailing, Coding, Meetings): Meetings

Rate the focus level required for "Client Meeting" on a scale of 1-5: 5

How often were you distracted during "Client Meeting" on a scale of 1-5: 1

Was the activity "Client Meeting" productive? (Yes/No): y

How would you evaluate the activity "Client Meeting"? (e.g., highly productive, could have been better): highly productive

How satisfied were you with the completion of "Client Meeting"? (e.g., very satisfied, somewhat satisfied, not satisfied): very satisfied

Please provide any additional thoughts on the activity "Client Meeting": Smooth

---------------------------------------------------------------------------------------------------------

Enter Category for activity "Report Writing" (e.g., Emailing, Coding, Meetings): Documentation

Rate the focus level required for "Report Writing" on a scale of 1-5: 4

How often were you distracted during "Report Writing" on a scale of 1-5: 2

Was the activity "Report Writing" productive? (Yes/No): Yes

How would you evaluate the activity "Report Writing"? (e.g., highly productive, could have been better): could have been better

How satisfied were you with the completion of "Report Writing"? (e.g., very satisfied, somewhat satisfied, not satisfied): somewhat satisfied

Please provide any additional thoughts on the activity "Report Writing": Smooth Writing

---------------------------------------------------------------------------------------------------------

Enter Category for activity "Team Collaboration" (e.g., Emailing, Coding, Meetings): Meetings

Rate the focus level required for "Team Collaboration" on a scale of 1-5: 5

How often were you distracted during "Team Collaboration" on a scale of 1-5: 5

Was the activity "Team Collaboration" productive? (Yes/No): no

How would you evaluate the activity "Team Collaboration"? (e.g., highly productive, could have been better): could have been better

How satisfied were you with the completion of "Team Collaboration"? (e.g., very satisfied, somewhat satisfied, not satisfied): not satisfied

Please provide any additional thoughts on the activity "Team Collaboration": -

---------------------------------------------------------------------------------------------------------

How was your overall task experience? : Amazing Day!
```

3.6.5 Data Validation for Focus Level

If the user enters an invalid number and is not in the range, the system will display "**Invalid Input. Please enter a number between 1 and 5:**" until the user enters a valid number.

```
--------------------
Productivity Tracking
--------------------

---------------------------------------------------------------------------------------------------------

Enter Category for activity "Help me" (e.g., Emailing, Coding, Meetings): Meeting

Rate the focus level required for "Help me" on a scale of 1-5: 8

Invalid Input. Please enter a number between 1 and 5: -1

Invalid Input. Please enter a number between 1 and 5:
```

3.6.6 Data Validation for Yes/No Question

If the user enters an invalid answer, the system will display "**Invalid input. Please answer (Y-Yes or N-No):**" until the user enters a valid number.

```
Was the activity "Help me" productive? (Yes/No): g
Invalid input. Please answer (Y-Yes or N-No): k
Invalid input. Please answer (Y-Yes or N-No): r
Invalid input. Please answer (Y-Yes or N-No): -
Invalid input. Please answer (Y-Yes or N-No):
```

3.6.7 Notification for Completion of Safety Checklist

Once the user has successfully answered the questions, the program will notify the user that the entry is completed and recorded to the **workLog.txt**.

Furthermore, the program will wait for the user input to continue the program.

```
------------------------------------------------------------

How was your overall task experience? : Amazing!

Work log saved successfully to WorkLog.txt!

Enter any key to continue...
```

3.7 Generate Productive Report Function

If the user enters option [2] to generate a Productive Report, the data recorded from **workLog.txt** will be read and displayed.

```
Report List

Generated Report:


WORK LOG - ENTRY 1
------------------------------------------------------------------------------------------------------------------
NAME                    CONTACT                  TODAY'S DATE        DEPARTMENT          SUPERVISOR
------------------------------------------------------------------------------------------------------------------
Aisyah Rahman           +60 12-345 6789          01/09/2024          Human Resources     Mr. Ahmad Sulaiman

------------------------------------------------------------------------------------------------------------------
ACTIVITY                                      START TIME  END TIME    TOTAL     STATUS          NOTES
------------------------------------------------------------------------------------------------------------------
Client Meeting                                00:00       02:00       02:00     Completed       The meeting went well
Report Writing                                02:00       05:00       03:00     In progress     Struggling
Team Collaboration                            08:00       10:00       02:00     Complete        Productive

------------------------------------------------------------------------------------------------------------------
Total Scheduled Time : 07:00
------------------------------------------------------------------------------------------------------------------


Productivity Tracking
------------------------------------------------------------------------------------------------------------------------------------------------------------------
ACTIVITY                      CATEGORY         FOCUS LEVEL   DISTRACTIONS   PRODUCTIVE?    EVALUATION                    SATISFACTION           FEEDBACK
------------------------------------------------------------------------------------------------------------------------------------------------------------------
Client Meeting                Client Meeting   5             1              Yes            highly productive             very satisfied         satisfied
Report Writing                Documentation    4             2              Yes            could have been better        somewhat satisfied     Smooth process
Team Collaboration            Meetings         5             5              No             could have been better         not satisfied         -

------------------------------------------------------------------------------------------------------------------------------------------------------------------
Productive Tasks     : 2
Non-Productive Tasks : 1
Total Number of Task Completed: 3
------------------------------------------------------------------------------------------------------------------------------------------------------------------
Overall Experience : Amazing
------------------------------------------------------------------------------------------------------------------------------------------------------------------


xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

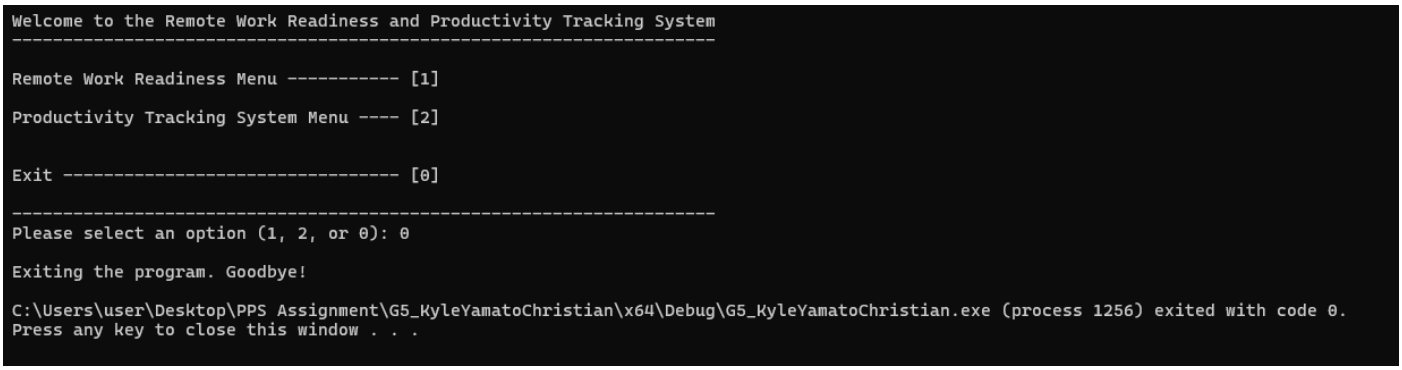
3.7.1 Validation of Generate Productivity Report Function

The program will display an error message if the user selects option [2] before completing option [1] and, will kindly ask the user to complete the **Log Work Activity & Track Productivity Metrics** before generating the report.

```
Posibble Reason 1: No Entry Recorded

Please Login your Work Activity and Complete Track Productivity Metrics (Option [1])

Before Generating Report


Enter any key to return...
```

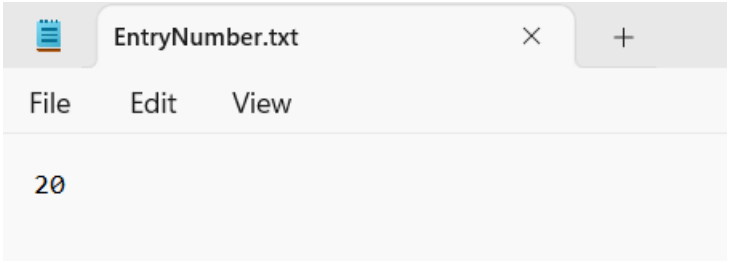
3.8 Exiting the program

When the user enters [0] in the main menu, the program will display an exit quote and close the program.
"**Exiting the program. Goodbye!**"



4 SAMPLE INPUT

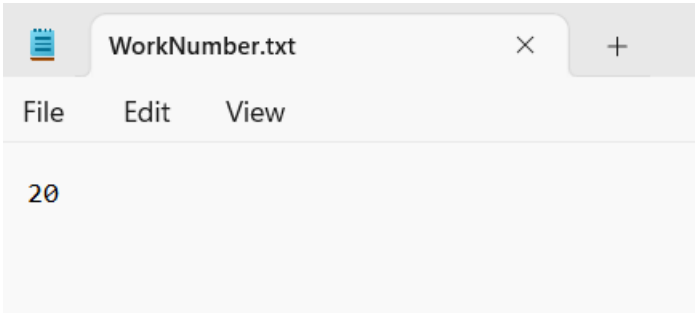
4.1 EntryNumber.txt



4.2 SafetyChecklist.txt

```
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

----------------------------------------------
Checklist Number: 2
----------------------------------------------

Home Office Safety Checklist

Employee                 : Leo
Date                     : 15/7/2024
Manager                  : Kyle
Remote work site address : No. 25, Lorong Bukit Raya, Taman Sejahtera, 43000 Kajang, Selangor
Description of work area : Frontend Development

----------------------------------------------------------------------------------------------------------------------
Question                                                                                                        Answer
----------------------------------------------------------------------------------------------------------------------

(1) Is the work space free from excessive noise?                                                                No

(2) Is adequate lighting (side or rear) provided at the work station?                                           Yes

(3) Is all electrical equipment free of recognized hazards that could cause physical harm
     (frayed wires running through walls, exposed wires fixed to the ceiling)?                                  Yes

(4) Is electrical system adequate for office equipment?                                                         Yes

(5) Is electrical equipment grounded?                                                                           Yes

(6) Are surge protectors properly installed?                                                                    Yes

(7) Are aisles, doorways and floors free of obstructions to permit visibility and movement?                     Yes

(8) Is there an exit that allows prompt exiting?                                                                Yes

(9) Are phone lines, electrical cords and extension wires secured under a desk or along a baseboard?            Yes

(10) Is the office space neat and clean?                                                                        Yes

(11) Is a working fire extinguisher located nearby?                                                             Yes

(12) Are working smoke detectors installed at the work site?                                                    Yes

(13) Is the work area private and free of intrusion?                                                            No

(14) Are files and data secure?                                                                                 No

(15) Are first aid supplies readily accessible and adequate?                                                    No

(16) Are office furniture and equipment ergonomically correct?                                                  Yes

(17) Desk: 29  high?                                                                                            Yes

(18) Chairs: Sturdy and adjustable (90  at knees, feet flat on floor, 15  back tilt)
     with backrest and casters appropriate for floor surface?                                                   Yes

(19) Keyboard: In line with wrist and forearm position?                                                         Yes

(20) Monitor: 20-14  from eyes: Top of screen slightly below eye level?                                         Yes

(21) Are work materials and equipment in a secure place that can be protected from damage or misuse?            No

(22) Are there security requirements in place to protect confidentiality and security of
     company information and computer systems?                                                                  No

----------------------------------------------------------------------------------------------------------------------
    Total number of Yes: 16
    Total number of No : 6
----------------------------------------------------------------------------------------------------------------------
    Summary: The workspace has several issues that should be addressed before it can be considered suitable.
----------------------------------------------------------------------------------------------------------------------

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

4.3 WorkNumber.txt



4.4 WorkLog.txt

WorkLog.txt

File Edit View

```
WORK LOG - ENTRY 1
------------------------------------------------------------------------------------------------------------------
NAME                    CONTACT                  TODAY'S DATE        DEPARTMENT          SUPERVISOR
------------------------------------------------------------------------------------------------------------------
Aisyah Rahman           +60 12-345 6789          01/09/2024          Human Resources     Mr. Ahmad Sulaiman

------------------------------------------------------------------------------------------------------------------
ACTIVITY                                     START TIME  END TIME    TOTAL     STATUS         NOTES
------------------------------------------------------------------------------------------------------------------
Client Meeting                               00:00       02:00       02:00     Complete       The meeting went well
Report Writing                               02:00       05:00       03:00     In Progress    Struggling
Team Collaboration                           08:00       10:00       02:00     Complete       Productive
System Maintenance                           11:00       12:00       01:00     Complete       Implemented smoothly
Training Session                             13:00       15:00       02:00     Not Started    -

------------------------------------------------------------------------------------------------------------------
Total Scheduled Time : 10:00
------------------------------------------------------------------------------------------------------------------


Productivity Tracking
------------------------------------------------------------------------------------------------------------------------------------------------------------
ACTIVITY                      CATEGORY         FOCUS LEVEL   DISTRACTIONS   PRODUCTIVE?    EVALUATION                    SATISFACTION          FEEDBACK
------------------------------------------------------------------------------------------------------------------------------------------------------------
Client Meeting                Meetings         5             1              Yes            highly productive             very satisfied        -
Report Writing                Documentation    5             1              Yes            highly productive             very satisfied        Smooth
Team Collaboration            Meetings         5             4              No             could have been better        not satisfied         easily distracted
System Maintenance            IT Support       5             5              Yes            highly productive             very satisfied        Smooth
Training Session              Training         3             3              Yes            highly productive             somewhat satisfied    -

------------------------------------------------------------------------------------------------------------------------------------------------------------
Productive Tasks      : 4
Non-Productive Tasks : 1
Total Number of Task Completed: 5
-------------------------------------------------------------------------------------------------------------------------------------------------------------
Overall Experience : Amazing!
-------------------------------------------------------------------------------------------------------------------------------------------------------------
```

WorkLog.txt

File Edit View

```
WORK LOG - ENTRY 2
------------------------------------------------------------------------------------------------------------------
NAME                    CONTACT                  TODAY'S DATE        DEPARTMENT          SUPERVISOR
------------------------------------------------------------------------------------------------------------------
Daniel Tan              +60 19-876 5432          02/09/2024          Finance             Ms. Nora Idris

------------------------------------------------------------------------------------------------------------------
ACTIVITY                                     START TIME  END TIME    TOTAL     STATUS         NOTES
------------------------------------------------------------------------------------------------------------------
Client Meeting                               00:00       02:00       02:00     Complete       The meeting went well
Report Writing                               02:00       05:00       03:00     In Progress    Struggling
Team Collaboration                           08:00       10:00       02:00     Complete       Productive
System Maintenance                           11:00       12:00       01:00     Complete       Implemented smoothly
Training Session                             13:00       15:00       02:00     Not Started    -
Market Research                              15:00       16:00       01:00     Complete       Data challenges
Product Development                          16:00       17:00       01:00     In Progress    Minor issues
Customer Support                             18:00       19:00       01:00     Complete       Resolved
Budget Planning                              20:00       21:00       01:00     Not Started    -
Content Creation                             21:00       23:00       02:00     Complete       Revisions needed

------------------------------------------------------------------------------------------------------------------
Total Scheduled Time : 16:00
------------------------------------------------------------------------------------------------------------------


Productivity Tracking
------------------------------------------------------------------------------------------------------------------------------------------------------------
ACTIVITY                      CATEGORY         FOCUS LEVEL   DISTRACTIONS   PRODUCTIVE?    EVALUATION                    SATISFACTION          FEEDBACK
------------------------------------------------------------------------------------------------------------------------------------------------------------
Client Meeting                Meetings         5             1              Yes            highly productive             very satisfied        -
Report Writing                Documentation    5             1              Yes            highly productive             very satisfied        Smooth
Team Collaboration            Meetings         5             4              No             could have been better        not satisfied         easily distracted
System Maintenance            IT Support       5             5              Yes            highly productive             very satisfied        Smooth
Training Session              Training         3             3              Yes            highly productive             somewhat satisfied    -
Market Research               Research         5             1              Yes            highly productive             very satisfied        -
Product Development           Development      5             1              Yes            highly productive             very satisfied        Smooth
Customer Support              Support          5             4              No             could have been better        not satisfied         easily distracted
Budget Planning               Planning         5             5              Yes            highly productive             very satisfied        Smooth
Content Creation              Creative         3             3              Yes            highly productive             somewhat satisfied    -

------------------------------------------------------------------------------------------------------------------------------------------------------------
Productive Tasks      : 8
Non-Productive Tasks : 2
Total Number of Task Completed: 10
-------------------------------------------------------------------------------------------------------------------------------------------------------------
Overall Experience : Fantastic!
-------------------------------------------------------------------------------------------------------------------------------------------------------------

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

5 CONTRIBUTION TABLE

Members	Contribution
1) Kyle Yamato Christian (2302452)	1) Report 2) C++ Code 3) Productivity Tracking 20 Samples
2) Tan Jun Yin (2302533)	1) Report (1.7 Special Issues/Constraints) 2) Flow chart (Part 2)
3) Ang Yong Xen (2301521)	1) Flow chart (Part 1) 2) Structure Chart
4) Lim Jun Sheng (2302532)	1) Pseudocode 2) Safety Checklist 20 Samples